OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 10

MANAGING PROCESSES

SUGGESTED TIME: 7 PERIODS

IN THIS CHAPTER

This chapter explains how an operating system creates, schedules and coordinates processes so that many programs appear to run simultaneously on a single processor. Students master scheduling algorithms, synchronisation tools and deadlock handling — core topics of the GCE Advanced Level Computer Science paper.

- By the end of this chapter I can define a process and distinguish it from a program
- By the end of this chapter I can describe the process states and draw the state transition diagram
- By the end of this chapter I can explain the role of the Process Control Block (PCB) and of context switching
- By the end of this chapter I can distinguish preemptive from non-preemptive scheduling
- By the end of this chapter I can compute turnaround time, waiting time and draw Gantt charts for FCFS, SJF, SRTF, Round Robin and priority scheduling
- By the end of this chapter I can compare scheduling algorithms using performance criteria and choose the appropriate one for a given situation

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

CONTENTS

1. Section 1: Process Management Basics (Lesson 44)
2. Section 2: Process Scheduling and Non-preemptive Algorithms (Lessons 45–46)
3. Section 3: Preemptive Scheduling (Lesson 47)
4. Section 4: Process Synchronisation (Lesson 48)
5. Section 5: Deadlock Handling and Integration (Lessons 49–50)
6. Integration activity
7. Methods and worked examples
8. Exercises
9. Solutions to the exercises
10. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can define a process and distinguish it from a program
- By the end of this chapter I can describe the process states and draw the state transition diagram
- By the end of this chapter I can explain the role of the Process Control Block (PCB) and of context switching
- By the end of this chapter I can distinguish preemptive from non-preemptive scheduling
- By the end of this chapter I can compute turnaround time, waiting time and draw Gantt charts for FCFS, SJF, SRTF, Round Robin and priority scheduling
- By the end of this chapter I can compare scheduling algorithms using performance criteria and choose the appropriate one for a given situation
- By the end of this chapter I can explain race conditions, critical sections and mutual exclusion
- By the end of this chapter I can use semaphores to solve classical synchronisation problems (producer–consumer)
- By the end of this chapter I can state the four deadlock conditions and apply prevention, avoidance (Banker's algorithm) and detection strategies

PREREQUISITES

- Role and functions of an operating system (Lower Sixth: introduction to system software)
- Basic hardware structure: CPU, registers, main memory, interrupts
- Multiprogramming and time-sharing concepts seen in the OS overview chapter
- Elementary algorithmics: queues (FIFO), averages and simple table computations
- Notion of concurrent users from the databases/networks chapters

1 SECTION 1: PROCESS MANAGEMENT BASICS (LESSON 44)

1.1 The Hook: A Mobile Money Kiosk in Douala

Imagine a busy mobile money kiosk in Douala. One agent serves dozens of customers: some transactions are quick airtime top-ups, others are long money transfers requiring multiple confirmations. If the agent serves strictly in arrival order, a single long transfer can make twenty customers wait; if the agent keeps interrupting customers to be "fair", some may never finish because of the constant restarting. Worse, imagine two customers each holding one of the two forms the other needs — nobody moves. A computer's processor faces exactly the same dilemma every millisecond with hundreds of competing **processes**. How does the operating system decide who runs, for how long, and how does it prevent total gridlock? This section builds the fundamental vocabulary and mechanisms the OS uses to answer those questions.

1.2 What Is a Process?

DEFINITION **Program vs. Process**

A **program** is a passive, static entity: a file containing executable code and data stored on secondary storage (e.g., `/bin/ls` or `chrome.exe`). A **process** is an active, dynamic entity: a program in execution. It includes the program code (text section), current values of variables (data section), the execution stack (function calls, local variables), the heap (dynamic memory), and the **process context** — the content of the CPU registers (program counter, stack pointer, general-purpose registers, status flags) at a given instant.

WARNING **Program ≠ Process**

Never confuse the executable file on disk with the running instance in memory. One program can give rise to many processes (e.g., opening three terminal windows runs three `bash` processes). Conversely, a process may load several shared libraries, but it remains a single execution flow.

DID YOU KNOW? **Why Processes? Multiprogramming and Time-Sharing**

Early computers ran one program at a time (batch processing). The CPU often sat idle waiting for slow I/O devices. **Multiprogramming** keeps several processes in memory simultaneously; when one process waits for I/O, the OS switches the CPU to another. **Time-sharing** (multitasking) extends this by rapidly switching the CPU among processes, giving each user the illusion of a dedicated machine. Both techniques maximise CPU utilisation

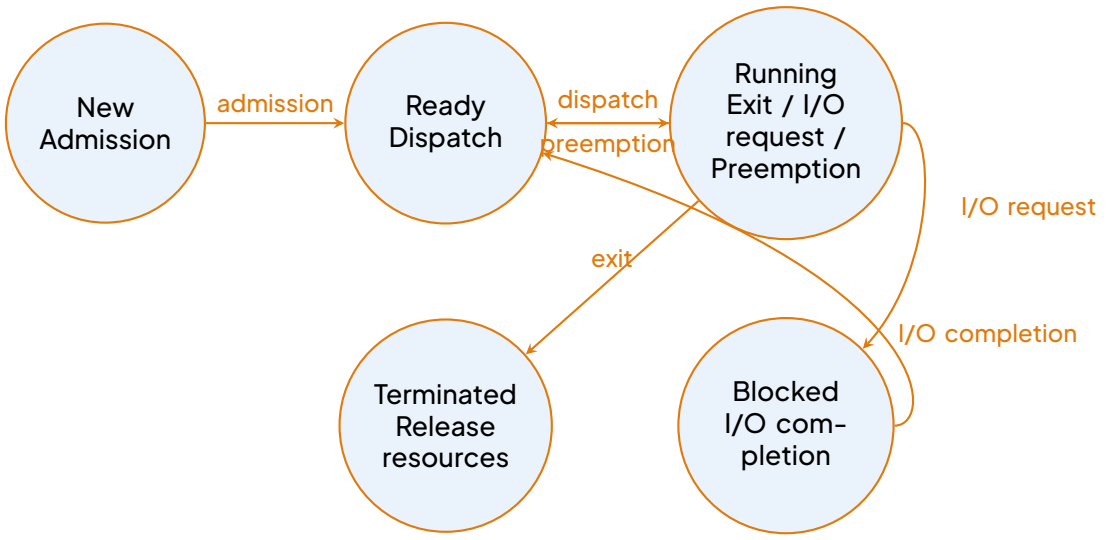
and throughput.

1.3 Process States and Transitions

At any moment a process occupies exactly one of five states. The OS moves processes between states in response to specific events.

DEFINITION **Five Process States**

- **New**: The process is being created; the OS allocates a PCB and initial resources.
- **Ready**: The process resides in main memory, has all resources except the CPU, and waits only for the dispatcher to assign it a processor core.
- **Running**: The process's instructions are currently executing on a CPU core.
- **Blocked** (or **Waiting**): The process cannot continue until an external event occurs (I/O completion, signal from another process, timer expiry, etc.). It voluntarily relinquishes the CPU.
- **Terminated** (or **Exit**): The process has finished execution (normal exit or forced kill). The OS reclaims resources but keeps the PCB briefly so the parent can read the exit status.



Process state transition diagram. Arrows show the event that triggers the transition.

METHOD **Identifying the State of a Process from a Scenario**

1. Locate the keyword: "waiting for keyboard input" → Blocked; "selected by the scheduler" → Running; "in memory, ready to run" → Ready.
2. Check whether the process holds the CPU. If yes → Running. If no, but it could run immediately if given CPU → Ready. If it needs an external event → Blocked.
3. Creation events (fork, exec) → New. Termination (exit, kill) → Terminated.

EXEMPLE **State Identification**

A user opens a text editor. The editor reads a configuration file (disk I/O), then waits for a keystroke. Meanwhile the OS runs a background antivirus scan.

- Editor during disk read: Blocked (waiting for I/O completion).
- Editor after file read, waiting for keystroke: Blocked (waiting for user input).
- Antivirus scan while editor is blocked: Running (has CPU).
- Editor after keystroke arrives, before scheduler picks it: Ready.

1.4 The Process Control Block (PCB)

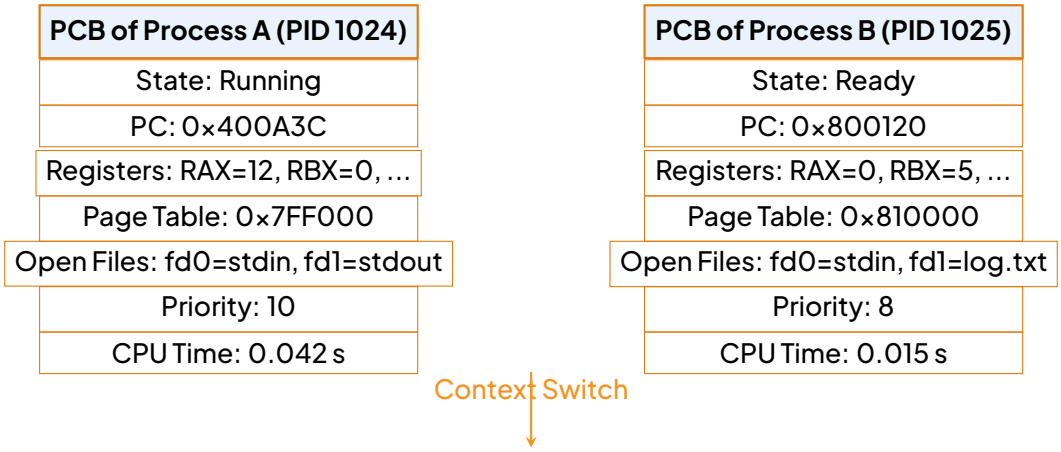
Every process is represented in the OS by a **Process Control Block** (PCB), sometimes called a **Task Control Block**. The PCB is the OS's "identity card" for the process.

DEFINITION **Process Control Block (PCB)**

A PCB is a kernel data structure that contains all information needed to suspend and later resume a process. Typical fields:

- Process ID (PID) — unique integer identifier.
- Process State — Ready, Running, Blocked, etc.
- Program Counter (PC) — address of the next instruction to execute.
- CPU Registers — general-purpose registers, stack pointer, status flags.
- Memory Management Info — page table pointer, segment table, memory limits.
- Open File Descriptors — list of files, sockets, pipes the process uses.
- Scheduling Information — priority, scheduling class, time quantum used.
- Accounting Information — CPU time used, real time elapsed, user/group IDs.

- I/O Status Information — devices allocated, pending I/O requests.



Two PCBs during a context switch from Process A (Running) to Process B (Ready).

1.5 Context Switching

DEFINITION **Context Switch**

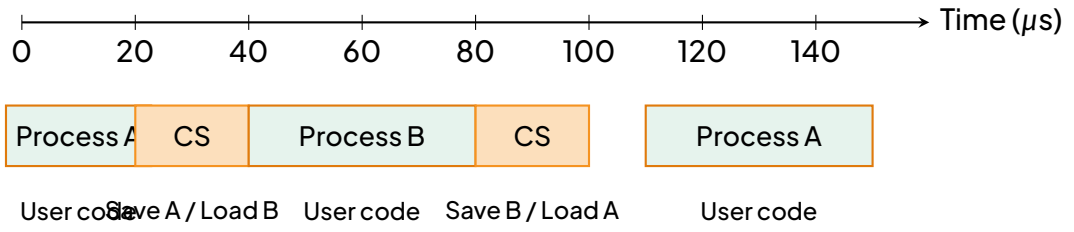
A **context switch** is the procedure of saving the context (CPU registers, PC, stack pointer, etc.) of the currently running process into its PCB and restoring the context of the next process to run from its PCB. It is triggered by interrupts (timer, I/O), system calls (blocking I/O, yield), or scheduler decisions.

PROPERTY **Context Switch Overhead**

A context switch performs no useful user work; it is pure OS overhead. Typical costs on modern hardware: 1–5 μs (microseconds) for saving/restoring registers, flushing pipelines, invalidating TLB entries, and updating kernel data structures. Excessive switching (thrashing) wastes CPU cycles.

WARNING **Context Switch Overhead**

Do not confuse context switching with the scheduler's decision-making. The scheduler chooses the next process; the dispatcher performs the context switch. Frequent preemption (e.g., a time quantum of 1 ms) increases overhead; too long a quantum hurts responsiveness. The OS designer must balance both.



Timeline showing CPU alternating between two processes with context-switch (CS) gaps. Time scale: each unit = 20 μs.

1.6 Process Creation and Termination

DEFINITION **Process Creation**

In Unix-like systems, a new process is created by the **fork()** system call. The parent process is cloned: the child receives a copy of the parent's address space (copy-on-write optimises this), file descriptors, and register state. The child gets a new PID. The return value of `fork()` distinguishes parent (> 0, child's PID) from child (= 0). The child often calls **exec()** to replace its memory image with a new program.

DEFINITION **Process Termination**

A process terminates via **exit()** (normal) or a signal (abnormal). The OS changes its state to Terminated, releases memory, closes files, and notifies the parent (via `SIGCHLD`). The parent can call **wait()** to read the child's exit status.

WARNING **Orphan and Zombie Processes**

- Orphan: A child whose parent has terminated. The OS re-parents it to `init` (PID 1), which periodically calls `wait()`.

- Zombie: A terminated child whose parent has not yet called `wait()`. The PCB remains in the process table, consuming a PID slot. Many zombies can exhaust the PID space.

1.7 Processes vs. Threads (GCE-Useful Extra)

DEFINITION **Thread**

A **thread** (lightweight process) is a single sequential flow of control within a process. Threads share the process's code, data, heap, and open files, but each has its own stack, program counter, and register set. Creating and switching threads is faster than processes because no memory-map changes are needed.

KEY POINTS **Process vs. Thread Summary**

Aspect	Process	Thread
Memory	Separate address spaces	Shared address space
Creation cost	High (copy page tables, etc.)	Low (allocate stack, TCB)
Context switch	Slow (TLB flush, new page table)	Fast (register swap only)
Communication	IPC (pipes, sockets, shared mem)	Direct via shared variables
Fault isolation	Crash affects only that process	Crash kills whole process

1.8 The Process Table

The OS maintains a **process table** (or task array), an array or linked list of pointers to all PCBs in the system. The index or pointer serves as the PID. The scheduler scans the process table to find Ready processes; the dispatcher uses the PCB pointer to load context. In Linux, the process table is the `task_struct` array; each entry is a `task_struct` (the PCB).

WORKED EXAMPLE 1 **Process Table Walkthrough**

Statement: A system has three processes: PID 1 (`init`), PID 1024 (text editor), PID 1025 (compiler). The editor is blocked on keyboard input; the compiler is running; `init` is ready. Show the process table entries and explain what happens when the user presses a key.
Solution:

PID	State	PC	Notes
1	Ready	0×8000	Parent of all orphans
1024	Blocked	0×400A3C	Waiting for keyboard interrupt
1025	Running	0×800120	Currently on CPU

When the key is pressed:

1. Keyboard controller raises interrupt → CPU vectors to keyboard ISR.
2. ISR places scan code in kernel buffer, signals the editor's wait queue.
3. OS changes editor's state from Blocked to Ready (updates PCB).
4. Scheduler may preempt compiler if editor has higher priority; otherwise editor waits in Ready queue.
5. Next context switch loads editor's PCB.

1.9 Worked Exercise: Identifying States and Transitions

EXERCISE 1 **State Transitions in a Print Job**

A user prints a large PDF. The print spooler process reads the file from disk, sends data to the printer via USB, and waits for the printer to signal "page done" before sending the next page. The CPU runs a video player while the spooler waits. Identify the state of the spooler at each stage and the transitions.

SOLUTION **Exercise 1**

1. Spooler starts (fork+exec): New $\xrightarrow{\text{admission}}$ Ready.
2. Scheduler dispatches spooler: Ready $\xrightarrow{\text{dispatch}}$ Running.
3. Spooler issues disk read: Running $\xrightarrow{\text{I/O request}}$ Blocked.
4. Disk interrupt, data ready: Blocked $\xrightarrow{\text{I/O completion}}$ Ready.
5. Spooler dispatched again: Ready $\xrightarrow{\text{dispatch}}$ Running.
6. Spooler sends page to printer, waits for "page done": Running $\xrightarrow{\text{I/O request}}$ Blocked.

7. Video player runs during wait: Running (video player).
8. Printer interrupt "page done": Blocked $\xrightarrow{\text{I/O completion}}$ Ready (spooler).
9. ... cycle repeats until last page.
10. Spooler calls exit: Running $\xrightarrow{\text{exit}}$ Terminated.

1.10 Key Points to Retain

KEY POINTS **Lesson 44 Essentials**

- A process is a program in execution; a program is a static file.
- Five states: New, Ready, Running, Blocked, Terminated. Know the six transitions: admission, dispatch, exit, I/O request, I/O completion, preemption.
- The PCB holds everything needed to stop and restart a process (PID, state, PC, registers, memory info, open files, scheduling info).
- Context switch = save current PCB + load next PCB. Pure overhead; keep it fast and infrequent enough.
- fork() creates a child (copy of parent); exec() loads a new program. exit() terminates; wait() reaps zombies.
- Threads share memory, lighter than processes; useful for parallelism within one application.
- The process table is the OS's directory of all PCBs.

2 SECTION 2: PROCESS SCHEDULING AND NON-PREEMPTIVE ALGORITHMS (LESSONS 45–46)

In the previous section we saw that a **process** is a program in execution, characterised by its **Process Control Block (PCB)** and moving through states such as New, Ready, Running, Waiting and Terminated. We now address a central question: **when several processes are in the Ready state, which one should be given the CPU next?** This decision is the job of the **CPU scheduler**, and the algorithms it uses determine the overall performance, responsiveness and fairness of the system.

2.1 Goals of CPU Scheduling

The scheduler aims to optimise several, often competing, criteria:

- **CPU utilisation** — keep the CPU as busy as possible (ideally 100%).
- **Throughput** — number of processes completed per unit time.
- **Turnaround time** — time from submission to completion (minimise).
- **Waiting time** — time spent in the ready queue (minimise).
- **Response time** — time from submission to first response (critical for interactive systems).
- **Fairness** — no process should starve indefinitely.

In batch systems, turnaround and throughput dominate; in interactive systems, response time is paramount. A good scheduler balances these metrics.

2.2 Key Time Metrics — Definitions and Formulas

Before studying algorithms, we must precisely define the time quantities used to evaluate them. Consider a process P_i with **arrival time** AT_i (when it enters the ready queue) and **burst time** BT_i (total CPU time it needs).

DEFINITION **Completion, Turnaround and Waiting Times**

For a process P_i:

- **Completion Time** CT_i — the clock time at which P_i finishes execution.
- **Turnaround Time** $TAT_i = CT_i - AT_i$ — total time in the system.
- **Waiting Time** $WT_i = TAT_i - BT_i$ — time spent waiting in the ready queue.

WARNING **Common Mistake: Wrong Waiting Time Formula**

Many candidates write $WT = CT - AT$. This is incorrect — that formula gives turnaround time. Waiting time excludes the actual CPU execution: $WT = TAT - BT = (CT - AT) - BT$. Always compute TAT first, then subtract BT.

2.3 The Scheduler, the Dispatcher and the Ready Queue

- **Ready queue** — a data structure (often a linked list or priority queue) holding PCBs of processes in the Ready state.
- **Short-term scheduler (CPU scheduler)** — selects a process from the ready queue and allocates the CPU. Invoked frequently (milliseconds), so must be fast.

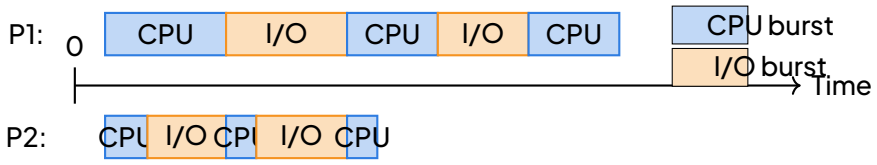
- **Dispatcher** — performs the **context switch**: saves the state of the outgoing process, loads the state of the incoming process, and jumps to its program counter. Dispatcher latency adds to scheduling overhead.
- **Long-term scheduler (job scheduler)** — decides which jobs are admitted to the ready queue from the job pool (controls degree of multiprogramming). Invoked rarely (seconds/minutes).
- **Medium-term scheduler** — swaps processes out to disk (suspended) and back in to manage memory pressure or improve process mix.

2.4 CPU-bound vs I/O-bound Processes and the CPU–I/O Burst Cycle

Processes alternate between **CPU bursts** (computation) and **I/O bursts** (waiting for devices).

- **CPU-bound process** — long CPU bursts, infrequent I/O (e.g., scientific computation, compilation).
- **I/O-bound process** — short CPU bursts, frequent I/O (e.g., text editor, web browser).

A good scheduler gives priority to I/O-bound processes: they release the CPU quickly, keeping I/O devices busy and improving response time for interactive users.



CPU–I/O burst cycle: CPU-bound process (P1) has long CPU bursts; I/O-bound process (P2) has short CPU bursts and frequent I/O.

2.5 Non-preemptive Scheduling — Definition

DEFINITION **Non-preemptive Scheduling**

In **non-preemptive scheduling**, once a process is allocated the CPU, it keeps the CPU until it terminates or blocks for I/O (enters the Waiting state). The scheduler cannot forcibly take the CPU away.

This simplifies the dispatcher (no forced context switches) but can lead to long waiting times for short processes if a long process is running — the **convoy effect**.

2.6 First-Come First-Served (FCFS) Scheduling

2.7 Rule and Intuition

FCFS is the simplest non-preemptive algorithm: the process that requests the CPU **first** gets it **first**. The ready queue is a FIFO queue. When the CPU becomes free, the process at the head

of the queue is dispatched.

2.8 Worked Example — FCFS Gantt Chart and Metrics

Consider four processes with the following arrival and burst times:

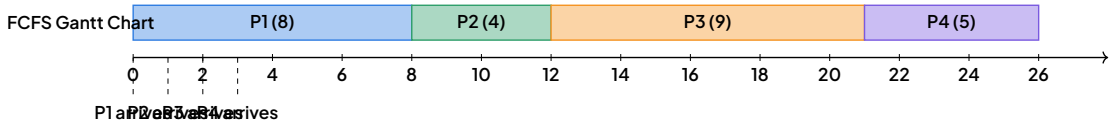
Process	Arrival Time AT	Burst Time BT
P1	0	8
P2	1	4
P3	2	9
P4	3	5

METHOD **Step-by-step: Solving Any Scheduling Exercise**

1. Draw the Gantt chart — simulate the algorithm on a timeline, respecting arrival times.
2. Read completion times CT_i from the Gantt chart (end of each process block).
3. Compute Turnaround Time $TAT_i = CT_i - AT_i$.
4. Compute Waiting Time $WT_i = TAT_i - BT_i$.
5. Calculate averages $\overline{TAT}$ and $\overline{WT}$.

WORKED EXAMPLE 2 **FCFS Scheduling for the Four Processes**

Step 1: Gantt Chart
Processes arrive at 0, 1, 2, 3. FCFS serves in arrival order: P1, P2, P3, P4.



FCFS Gantt chart: processes run in arrival order P1 → P2 → P3 → P4.

Step 2: Completion Times $CT_1 = 8$, $CT_2 = 12$, $CT_3 = 21$, $CT_4 = 26$.
Step 3: Turnaround Times $TAT_1 = 8-0 = 8$ $TAT_2 = 12-1 = 11$ $TAT_3 = 21-2 = 19$ $TAT_4 = 26-3 = 23$
Step 4: Waiting Times $WT_1 = 8-8 = 0$ $WT_2 = 11-4 = 7$ $WT_3 = 19-9 = 10$ $WT_4 = 23-5 = 18$
Step 5: Averages $\overline{TAT} = \frac{8+11+19+23}{4} = \frac{61}{4} = 15.25$ $\overline{WT} = \frac{0+7+10+18}{4} = \frac{35}{4} = 8.75$

Process	*AT*	*BT*	*CT*	*TAT*	*WT*
P1	0	8	8	8	0
P2	1	4	12	11	7
P3	2	9	21	19	10
P4	3	5	26	23	18
Average				15.25	8.75

2.9 The Convoy Effect

WARNING **Convoy Effect in FCFS**

If a long CPU-bound process (e.g., P1 with $BT = 8$) arrives first, all subsequent short processes (P2, P3, P4) must wait for it to finish, even though they could have completed quickly. This **convoy effect** inflates average waiting time and reduces CPU/I/O overlap. FCFS is therefore poor for interactive systems.

2.10 Shortest Job First (SJF) Scheduling

2.11 Rule and Intuition

SJF selects the process with the **smallest next CPU burst time** from the ready queue. It is non-preemptive: once started, a process runs to completion. SJF is **provably optimal** for minimising average waiting time for a given set of processes.

PROPERTY **Optimality of SJF (Non-preemptive)**

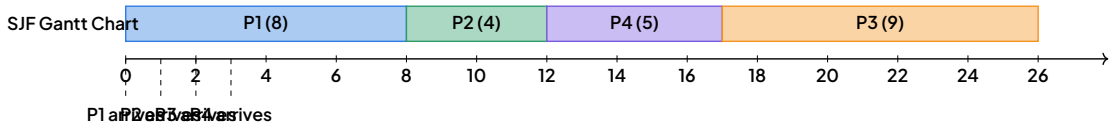
For any set of processes with known burst times, non-preemptive SJF gives the minimum possible average waiting time.

Proof sketch (examinable): Consider any schedule not in SJF order. There exists a pair of adjacent processes where a longer job precedes a shorter one. Swapping them reduces the waiting time of the shorter job by the burst time of the longer job, while increasing the waiting time of the longer job by the burst time of the shorter job. Since the shorter burst is smaller, the net change in total waiting time is negative. By repeated swaps, any schedule can be transformed into SJF order while strictly decreasing average waiting time. Hence SJF is optimal.

2.12 Worked Example — SJF on the Same Four Processes

WORKED EXAMPLE 3 **SJF Scheduling for the Four Processes**

Step 1: Gantt Chart
At time 0, only P1 is available → P1 runs 0–8. At time 8, ready queue: P2 (BT = 4), P3 (BT = 9), P4 (BT = 5). Shortest is P2 → runs 8–12. At time 12, ready: P3 (9), P4 (5). Shortest is P4 → runs 12–17. At time 17, only P3 remains → runs 17–26.



SJF Gantt chart: at each decision point, the shortest available burst is chosen. Order: P1 → P2 → P4 → P3.

Step 2: Completion Times $CT_1 = 8, CT_2 = 12, CT_4 = 17, CT_3 = 26$.
Step 3: Turnaround Times $TAT_1 = 8-0 = 8$ $TAT_2 = 12-1 = 11$ $TAT_4 = 17-3 = 14$ $TAT_3 = 26-2 = 24$
Step 4: Waiting Times $WT_1 = 8-8 = 0$ $WT_2 = 11-4 = 7$ $WT_4 = 14-5 = 9$ $WT_3 = 24-9 = 15$
Step 5: Averages $\overline{TAT} = \frac{8+11+14+24}{4} = \frac{57}{4} = 14.25$ $\overline{WT} = \frac{0+7+9+15}{4} = \frac{31}{4} = 7.75$

Process	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	1	4	12	11	7
P4	3	5	17	14	9
P3	2	9	26	24	15
Average				14.25	7.75

Comparison: SJF reduces average waiting time from 8.75 (FCFS) to 7.75, and average turnaround from 15.25 to 14.25.

2.13 Problems with SJF

- **Predicting burst times:** The OS does not know BT in advance. Common heuristic: **exponential averaging** $\tau_{n+1} = \alpha \cdot t_n + (1-\alpha)\tau_n$, where t_n is the last actual burst, τ_n the predicted burst, $\alpha \in [0,1]$ (typically 0.5).
- **Starvation of long jobs:** A steady stream of short jobs can indefinitely postpone a long job. Remedy: **ageing** (increase priority of waiting processes over time).

2.14 Priority Scheduling (Non-preemptive Version)

2.15 Rule

Each process has a **priority** (integer; lower number = higher priority or vice versa — convention

must be stated). The scheduler picks the highest-priority process from the ready queue. Non-preemptive: runs to completion or block.

2.16 Worked Example — Priority Scheduling

Same four processes, with priorities (lower = higher priority):

Process	*AT*	*BT*	Priority
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2

WORKED EXAMPLE 4 **Non-preemptive Priority Scheduling**

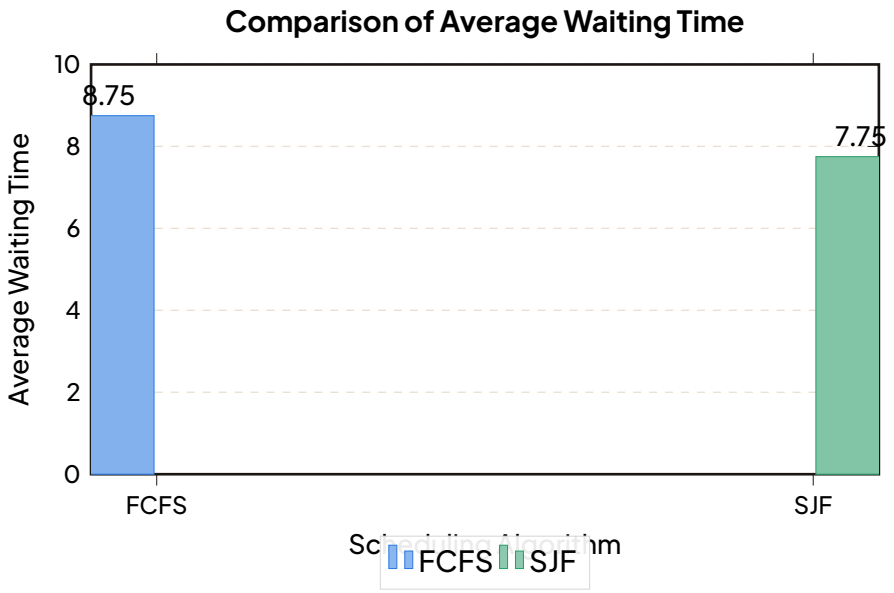
At time 0: only P1 (pri 3) → runs 0–8. At time 8: ready: P2 (pri 1), P3 (pri 4), P4 (pri 2). Highest priority = P2 → runs 8–12. At time 12: ready: P3 (pri 4), P4 (pri 2). Highest = P4 → runs 12–17. At time 17: P3 runs 17–26.
Order: P1 → P2 → P4 → P3 (same as SJF in this case, but for different reason).
Metrics identical to SJF example above: $\overline{WT} = 7.75$, $\overline{TAT} = 14.25$.

2.17 Starvation and Ageing

WARNING **Starvation in Priority Scheduling**

Low-priority processes may never run if high-priority processes keep arriving. **Ageing** gradually increases the priority of waiting processes (e.g., increment priority by 1 every fixed time interval). This guarantees eventual execution.

2.18 Comparative Visualisation: FCFS vs SJF Average Waiting Time



Bar chart comparing average waiting time: SJF (7.75) outperforms FCFS (8.75) for the given process set.

2.19 Summary of Non-preemptive Algorithms

Algorithm	Decision Rule	Advantages	Disadvantages
FCFS	Arrival order	Simple, fair (no starvation)	Convoy effect, high avg waiting time
SJF	Shortest burst first	Optimal avg waiting time	Requires burst prediction; starvation of long jobs
Priority	Highest priority first	Flexible, matches business needs	Starvation of low priority; needs ageing

2.20 Key Points to Remember

KEY POINTS **Key Points for Examination**

- Turnaround Time $TAT = CT - AT$; Waiting Time $WT = TAT - BT = CT - AT - BT$.
- Non-preemptive: process keeps CPU until termination or I/O block.
- FCFS: FIFO queue; suffers from convoy effect.
- SJF: picks shortest burst; provably minimises average waiting time (proof by pairwise

swap).

- SJF needs burst prediction (exponential averaging); can starve long jobs.
- Priority scheduling: highest priority first; starvation solved by ageing.
- Always draw Gantt chart first, then compute CT, TAT, WT, then averages.

2.21 Exercises

EXERCISE 2 **Practice: FCFS vs SJF**

Five processes arrive at times and have burst times as follows: P1: $AT = 0, BT = 10$; P2: $AT = 1, BT = 1$; P3: $AT = 2, BT = 2$; P4: $AT = 3, BT = 1$; P5: $AT = 4, BT = 5$.

1. Draw the FCFS Gantt chart and compute $\overline{WT}$ and $\overline{TAT}$.
2. Draw the SJF Gantt chart and compute $\overline{WT}$ and $\overline{TAT}$.
3. Explain the difference.

EXERCISE 3 **Priority with Ageing**

A system uses non-preemptive priority scheduling (lower number = higher priority). Processes: P1: $AT = 0, BT = 8, Pri = 3$; P2: $AT = 1, BT = 4, Pri = 1$; P3: $AT = 2, BT = 9, Pri = 4$; P4: $AT = 3, BT = 5, Pri = 2$. Ageing increases priority by 1 every 5 time units of waiting.

1. Simulate the schedule up to time 20.
2. Does ageing change the order compared to no ageing?

SOLUTION **Exercise 1**

FCFS: Order P1(0–10), P2(10–11), P3(11–13), P4(13–14), P5(14–19). CT: 10,11,13,14,19. TAT: 10,10,11,11,15. WT: 0,9,9,10,10. $\overline{WT} = 7.6$, $\overline{TAT} = 11.4$.
SJF: Time 0: P1 runs 0–10. Time 10: ready P2(1),P3(2),P4(1),P5(5). Tie P2/P4 (both 1) → FCFS tie-break: P2(10–11), P4(11–12), P3(12–14), P5(14–19). CT: P1=10, P2=11, P4=12, P3=14, P5=19. TAT: 10,10,9,12,15. WT: 0,9,8,10,10. $\overline{WT} = 7.4$, $\overline{TAT} = 11.2$. SJF slightly better; convoy effect visible in FCFS (P1 blocks many short jobs).

SOLUTION **Exercise 2**

Without ageing: same as priority example above (P1,P2,P4,P3). With ageing: at time 8, P3 waited 6 units → priority 4→3; P4 waited 5 units → priority 2→1. At time 8, ready: P2(pri1), P3(pri3), P4(pri1). Tie P2/P4 → FCFS: P2 runs. At time 12, P3 waited 10 units → priority 4→2; P4 waited 9 units → priority still 1 (next increment at 10 units waited, i.e., time 13). Ready: P3(pri2), P4(pri1). Highest priority is P4 → P4 runs. Order unchanged here, but ageing can change order in other cases.

DID YOU KNOW? **Real-world Scheduling in Linux**

Linux uses the **Completely Fair Scheduler (CFS)**, a preemptive, priority-based scheduler that models an ideal multi-tasking processor using virtual runtime. It approximates SJF for interactive tasks via dynamic priority adjustments. The concepts you learn here (FCFS, SJF, priority, ageing) are the building blocks of modern schedulers like CFS, Windows' Multi-level Feedback Queue, and real-time schedulers (Rate Monotonic, EDF) used in embedded systems controlling traffic lights in Yaoundé or Douala.

3 SECTION 3: PREEMPTIVE SCHEDULING (LESSON 47)

In the previous section we studied non-preemptive scheduling: once a process is given the CPU, it keeps it until it terminates or voluntarily enters the waiting state (e.g., for I/O). That policy is simple to implement, but it has a serious weakness: a single long process can monopolise the processor while many short, interactive processes wait in the ready queue. Imagine a cybercafé in Yaoundé where one customer is printing a 200-page document: if the attendant refuses to serve anyone else until the printing is over, every other customer leaves frustrated. A wiser attendant pauses the big job, serves the quick requests, then resumes the big job. This is exactly the idea of **preemptive scheduling**.

3.1 The Principle of Preemption

DEFINITION **Preemptive Scheduling**

A scheduling policy is **preemptive** if the operating system may interrupt a running process before it has finished its CPU burst, move it back to the ready queue, and allocate the CPU to another process. The interruption is usually triggered by a **timer interrupt** (the hardware clock signals the kernel at regular intervals, e.g., every 10–100 ms) or by the arrival of a higher-priority process.

When the CPU is taken away from a process, the operating system performs a **context switch**:

it saves the state of the interrupted process (program counter, CPU registers, memory management information) in its Process Control Block (PCB), then loads the state of the newly selected process. Context switches make preemption possible, but they are pure overhead: during a context switch the CPU does no useful work for any user process.

PROPERTY **Advantages and Costs of Preemption**

- Advantages: good **responsiveness** (interactive programs react quickly); no process can monopolise the CPU; essential for multi-user, time-sharing, and real-time systems.
- Costs: **context-switch overhead** at every preemption (saving/restoring registers, flushing pipelines, cache pollution); shared data may be left in an inconsistent state, so preemption creates the need for **synchronisation** mechanisms (studied in Section 4).

3.2 Shortest Remaining Time First (SRTF)

SRTF is simply the preemptive version of Shortest Job First (SJF). The rule is:

> Whenever a new process arrives (or a process finishes), compare the **remaining burst time** of the currently running process with the burst times of all processes in the ready queue. The process with the **shortest remaining time** gets the CPU — even if this means interrupting the running process.

WARNING **The Classic SRTF Mistake**

The most frequent examination error is to choose the shortest job once at the start and then let it run to completion. That is SJF, not SRTF! In SRTF you must re-evaluate the shortest remaining time at every new arrival. Draw a vertical line on your time axis at each arrival time and ask: "Does the newcomer beat the current process's remaining time?"

EXEMPLE **SRTF on Four Processes**

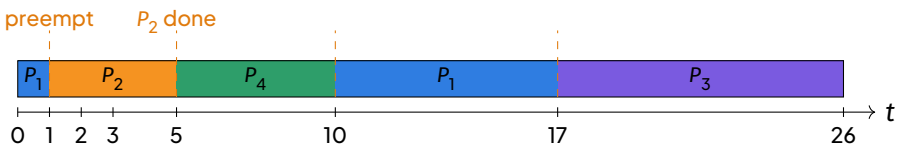
Consider the following processes (arrival time, burst time in ms):

Process	Arrival	Burst
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Trace of the scheduler (decision points at every arrival and completion):

- $t = 0$: only P_1 is present; it runs.
- $t = 1$: P_2 arrives (burst 4). P_1 has $8 - 1 = 7$ ms remaining. Since $4 < 7$, P_1 is preempted and P_2 runs.
- $t = 2$: P_3 arrives (9). P_2 has 3 remaining; $3 < 9$, so P_2 continues.
- $t = 3$: P_4 arrives (5). P_2 has 2 remaining; $2 < 5$, so P_2 continues.
- $t = 5$: P_2 finishes. Ready queue: P_4 (5), P_1 (7), P_3 (9). P_4 runs.
- $t = 10$: P_4 finishes. P_1 (7) beats P_3 (9); P_1 runs to completion at $t = 17$, then P_3 runs from 17 to 26.

Turnaround times (completion − arrival): $P_1 : 17$, $P_2 : 4$, $P_3 : 24$, $P_4 : 7$. Average turnaround $= \dfrac{17 + 4 + 24 + 7}{4} = 13$ ms.



Gantt chart for SRTF. P_1 is preempted at $t = 1$ when the shorter job P_2 arrives.

3.3 Round Robin (RR)

SRTF is optimal for average waiting time (provably minimal), but it requires knowing burst times in advance — which is rarely possible — and it can starve long processes. **Round Robin** takes a completely different, beautifully fair approach: every process gets a small, equal slice of the CPU, in turn.

DEFINITION **Time Quantum**

In Round Robin scheduling, the **time quantum** (or time slice) q is the maximum continuous time a process may hold the CPU before being preempted. The ready queue is organised as a circular FIFO queue: the scheduler takes the process at the front, lets it run for at most q time units, and if it is not finished, puts it back at the rear of the queue.

METHOD **Applying Round Robin — Step by Step**

1. Initialisation: Place all processes that have arrived at time 0 in the ready queue (FCFS

order). Set current time $t = 0$.

2. Event Loop: While the ready queue is not empty:
   (a) Handle arrivals at time t: Add any process arriving exactly at time t to the rear of the ready queue.
   (b) Handle preemption at time t (if $t > 0$): If the process that just finished its quantum (or blocked) is not terminated, move it to the rear of the ready queue.
   (c) Dispatch: Remove the process at the front of the queue. Let it run for $\Delta t = \min(q, \text{remaining burst})$. Advance time: $t \leftarrow t + \Delta t$.
   (d) Record: Add the interval to the Gantt chart. If the process finishes, record its completion time; otherwise it will be re-queued at the next iteration (step 2b).
3. Metrics: From the Gantt chart, compute for each process:
   - Turnaround Time TAT = Completion − Arrival
   - Waiting Time $WT = TAT$ − Burst

Convention used in this course: at a given instant t, arrivals are enqueued before the preempted process is re-queued. This ensures a newly arrived process gets a chance slightly sooner than the process that just used the CPU.

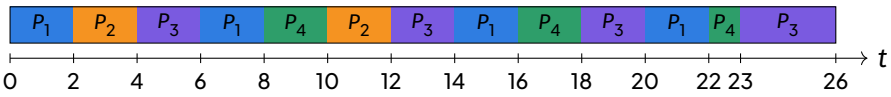
EXEMPLE **Round Robin with $q = 2$ on the Same Processes**

Using $P_1(0, 8)$, $P_2(1, 4)$, $P_3(2, 9)$, $P_4(3, 5)$ with quantum $q = 2$:

- $t = 0$: Queue $[P_1]$. P_1 runs 0–2 (rem 6).
- $t = 2$: Arrivals: P_3 (arr 2). Queue $[P_2, P_3]$. Preempt $P_1 \rightarrow$ Queue $[P_2, P_3, P_1]$. Dispatch P_2.
- $t = 4$: P_2 runs 2–4 (rem 2). Arrival: none. Preempt $P_2 \rightarrow$ Queue $[P_3, P_1, P_2]$. Dispatch P_3.
- $t = 6$: P_3 runs 4–6 (rem 7). Arrival: P_4 (arr 3, already in queue since $t = 3$? No, P_4 arrives at 3. At $t = 3$, P_2 was running. P_4 added to queue $\rightarrow [P_3, P_1, P_4]$? Wait. Let's trace queue at $t = 3$: Running P_2. Queue $[P_3, P_1]$. P_4 arrives $\rightarrow [P_3, P_1, P_4]$. At $t = 4$, Preempt $P_2 \rightarrow [P_3, P_1, P_4, P_2]$. Dispatch P_3.)
- $t = 6$: P_3 preempted (rem 7). Queue $[P_1, P_4, P_2, P_3]$. Dispatch P_1.
- $t = 8$: P_1 runs 6–8 (rem 4). Preempt $P_1 \rightarrow [P_4, P_2, P_3, P_1]$. Dispatch P_4.
- $t = 10$: P_4 runs 8–10 (rem 3). Preempt $P_4 \rightarrow [P_2, P_3, P_1, P_4]$. Dispatch P_2.
- $t = 12$: P_2 runs 10–12 and finishes. Queue $[P_3, P_1, P_4]$. Dispatch P_3.
- $t = 14$: P_3 runs 12–14 (rem 5). Preempt $P_3 \rightarrow [P_1, P_4, P_3]$. Dispatch P_1.

- $t = 16$: P_1 runs 14–16 (rem 2). Preempt $P_1 \rightarrow [P_4, P_3, P_1]$. Dispatch P_4.
- $t = 18$: P_4 runs 16–18 (rem 1). Preempt $P_4 \rightarrow [P_3, P_1, P_4]$. Dispatch P_3.
- $t = 20$: P_3 runs 18–20 (rem 3). Preempt $P_3 \rightarrow [P_1, P_4, P_3]$. Dispatch P_1.
- $t = 22$: P_1 runs 20–22 and finishes. Queue $[P_4, P_3]$. Dispatch P_4.
- $t = 23$: P_4 runs 22–23 and finishes. Queue $[P_3]$. Dispatch P_3.
- $t = 26$: P_3 runs 23–26 and finishes.

Turnarounds: $P_1 : 22$, $P_2 : 11$, $P_3 : 24$, $P_4 : 20$; average $= \frac{77}{4} = 19.25$ ms. The average is worse than SRTF, but notice that every process received CPU time within the first 10 ms — excellent responsiveness.



Gantt chart for Round Robin with quantum $q = 2$ on the same four processes.

PROPERTY **Bounded Response Time in RR**

With n processes in the ready queue and quantum q, a process waits at most $(n - 1) \times q$ time units before receiving the CPU again. Round Robin therefore guarantees a bounded response time: no process can starve. (This bound follows because at most $n - 1$ other processes can run for at most q each before your turn comes again.) The proof idea is examinable as a short justification, not a formal proof.

3.4 Choosing the Quantum

The performance of Round Robin depends entirely on the choice of q:

- **Very large quantum** ($q \geq$ longest CPU burst): no process is ever preempted, so RR degenerates into **FCFS**. Responsiveness is lost.
- **Very small quantum**: the scheduler context-switches constantly; if a context switch costs about the same as the quantum, most CPU time is wasted on administration instead of useful work (thrashing).
- **Reasonable quantum**: typically chosen so that the large majority of interactive bursts complete within one slice (real systems use values around 10–100 ms, e.g., Linux default time slice is dynamic but often in this range).

WARNING **Quantum Too Small**

A classic trap: candidates write "smaller quantum = fairer = better". Wrong. If the quantum is too small, **context-switch overhead dominates** and the effective CPU throughput collapses. In calculations, if the question gives a context-switch cost (e.g., 1 ms), you must add it to the timeline at every preemption (i.e., the clock advances by q + switch_cost for a full slice).

3.5 Preemptive Priority Scheduling, Starvation and Ageing

In **preemptive priority scheduling**, each process has a priority number (integer); the CPU always goes to the ready process with the **highest priority** (lowest number or highest number, convention must be stated), and a newly arrived high-priority process immediately preempts the running one. This is ideal for urgent tasks, but it creates a danger:

DEFINITION **Starvation and Ageing**

Starvation (or indefinite blocking) occurs when a low-priority process waits forever because higher-priority processes keep arriving. The standard cure is **ageing**: the priority of a waiting process is gradually increased (or its priority number decreased) as time passes, so that every process eventually reaches the top of the queue and runs.

WARNING **Exam Tip: State Your Tie-Breaking Rule**

Whenever two processes have equal priority, equal remaining time, or arrive simultaneously, a tie-breaking rule is needed. Always state it explicitly in your answer — the standard convention is FCFS (the process that arrived first goes first). An unstated assumption can cost you marks even if your Gantt chart is otherwise correct.

3.6 Multilevel Queue Scheduling (GCE-Useful Extra)

Real systems rarely use a single algorithm for all processes. Two hybrid schemes are worth knowing for the "integration activity" and essay questions:

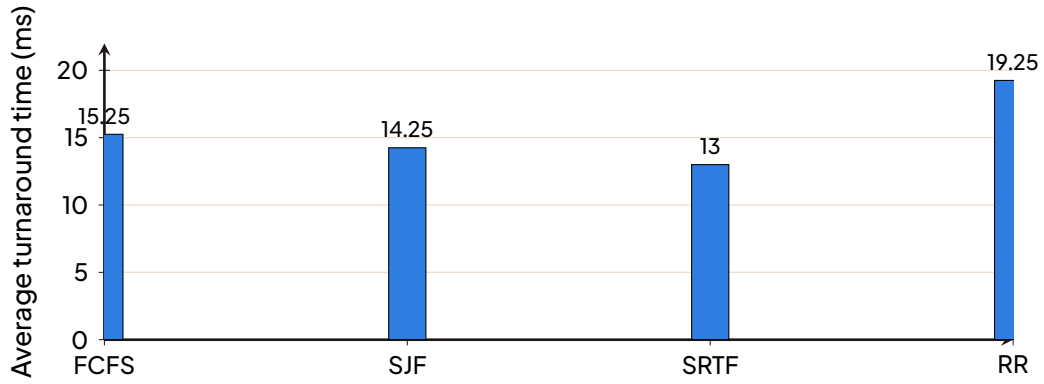
- **Multilevel queue:** the ready queue is split into several separate queues (e.g., system processes, interactive, batch), each with its own algorithm (RR for interactive, FCFS for batch). A process stays in its assigned queue forever, and queues are served by fixed priority (e.g., system > interactive > batch). Simple but rigid.
- **Multilevel feedback queue:** processes may move between queues. A new process starts in the top queue with a small quantum; if it uses its whole quantum (CPU-bound), it is demoted to a lower queue with a larger quantum; if it waits a long time, it is promoted (ageing). This adapts automatically to each process's behaviour and is the basis of many real

schedulers (e.g., classic Unix, Windows NT/XP).

DID YOU KNOW? **Real-World Schedulers**

Modern OS kernels use sophisticated variants of multilevel feedback queues. Linux's Completely Fair Scheduler (CFS) uses a red-black tree keyed by virtual runtime (effectively a dynamic priority) to approximate ideal fair sharing. Windows uses a priority-based preemptive scheduler with 32 priority levels and dynamic priority boosting for I/O completion and foreground windows. Both implement the core ideas you have learnt: preemption, time slices, priority, and ageing/boosting.

3.7 Comparing All the Scheduling Algorithms



Average turnaround time for the four algorithms on our example (P_1–P_4). SRTF wins on average, but RR gives the best responsiveness.

Algorithm	Preemptive?	Starvation risk	Overhead	Best use
FCFS	No	No	Very low	Simple batch systems
SJF	No	Yes (long jobs)	Low	Batch with known bursts
SRTF	Yes	Yes (long jobs)	Moderate	Batch, optimal average wait
RR	Yes	No	High if q small	Interactive / time-sharing
Priority	Both	Yes (cured by ageing)	Low	Real-time, mixed urgency

KEY POINTS **Key Points of Lesson 47**

- Preemption = the OS may interrupt a running process (timer interrupt) and reassign the CPU; it costs a context switch each time.
- SRTF = preemptive SJF: re-evaluate the shortest remaining time at every arrival.
- RR = circular FIFO with quantum q; unfinished processes go to the back of the queue; response time bounded by $(n-1)q$.
- q too large ⇒ behaves like FCFS; q too small ⇒ overhead dominates (thrashing).
- Preemptive priority can starve low-priority processes; ageing is the cure.
- Multilevel feedback queues combine the advantages of RR (responsiveness) and priority (urgency) with automatic adaptation.

WORKED EXAMPLE 5 **SRTF with a Late Arrival**

Processes: A (arrival 0, burst 6), B (arrival 2, burst 3), C (arrival 4, burst 2). Give the Gantt chart under SRTF and the average turnaround time.

SOLUTION

- $t = 0$: only A; it runs.
- $t = 2$: B arrives (3). A has $6 - 2 = 4$ left; $3 < 4$, so A is preempted, B runs.
- $t = 4$: C arrives (2). B has $3 - 2 = 1$ left; $1 < 2$, so B continues and finishes at $t = 5$.
- $t = 5$: choose between C (2) and A (4): C runs 5–7, then A runs 7–11.

Turnarounds: $A: 11 - 0 = 11$, $B: 5 - 2 = 3$, $C: 7 - 4 = 3$. Average $= \frac{11 + 3 + 3}{3} = \frac{17}{3} \approx 5.67$ ms.

EXPERIMENT **Simulating a Scheduler (Guided Mini-Project)**

Write a short Python script (or pseudocode) that simulates the RR scheduler for a given list of processes (arrival, burst) and a quantum q. Use a list as a queue (`pop(0)` for dequeue, `append()` for enqueue). Print the Gantt chart as a list of tuples `(pid, start, end)`. Test it with the examples from this section. This reinforces the exact queue mechanics and helps avoid off-by-one errors in exams.

EXERCISE 4 **Practice**

Using $P_1(0, 7)$, $P_2(2, 4)$, $P_3(4, 1)$, $P_4(5, 4)$: (a) Draw the SRTF Gantt chart and compute the average waiting time. (b) Draw the Round Robin Gantt chart with $q = 2$ (use the arrival-first convention from the Method box) and compute the average waiting time. (c) State which algorithm gives the better average waiting time and which gives the better first response time for P_3.

SOLUTION **Exercise 1**

(a) SRTF Trace:

- $t = 0$: P_1 runs (rem 7).
- $t = 2$: P_2 arrives (4). P_1 rem 5. $4 < 5 \rightarrow$ Preempt P_1, Run P_2.
- $t = 4$: P_3 arrives (1). P_2 rem 2. $1 < 2 \rightarrow$ Preempt P_2, Run P_3.
- $t = 5$: P_3 finishes. Ready: $P_2(2)$, $P_1(5)$, $P_4(4)$ [arrives at 5]. Shortest $P_2(2)$ runs.
- $t = 7$: P_2 finishes. Ready: $P_4(4)$, $P_1(5)$. P_4 runs.
- $t = 11$: P_4 finishes. P_1 runs 11-16.

Gantt: $P_1(0-2)$, $P_2(2-4)$, $P_3(4-5)$, $P_2(5-7)$, $P_4(7-11)$, $P_1(11-16)$. Waiting: $P_1 = (2-0)+(11-7) = 2 + 4 = 6$? No. P_1 runs 0–2, waits 2–11 (9), runs 11–16. Wait=9. P_2: waits 4–5 (1). Wait=1. P_3: wait=0. P_4: arrives 5, starts 7. Wait=2. Avg Wait = $(9 + 1 + 0 + 2)/4 =$ **3 ms**.

(b) RR $q = 2$ Trace (Arrivals first, then Preemption):

- $t = 0$: Queue $[P_1]$. Run P_1 0-2 (rem 5).
- $t = 2$: Arr P_2. Queue $[P_1, P_2]$. Preempt $P_1 \rightarrow [P_2, P_1]$. Run P_2 2-4 (rem 2).
- $t = 4$: Arr P_3. Queue $[P_1, P_3]$. Preempt $P_2 \rightarrow [P_1, P_3, P_2]$. Run P_1 4-6 (rem 3).
- $t = 5$: Arr P_4. Running P_1. Queue $[P_3, P_2, P_4]$.
- $t = 6$: Preempt $P_1 \rightarrow [P_3, P_2, P_4, P_1]$. Run P_3 6-7 (finishes, burst 1).
- $t = 7$: Queue $[P_2, P_4, P_1]$. Run P_2 7-9 (finishes, rem 2).
- $t = 9$: Queue $[P_4, P_1]$. Run P_4 9-11 (rem 2).
- $t = 11$: Preempt $P_4 \rightarrow [P_1, P_4]$. Run P_1 11-13 (rem 1).
- $t = 13$: Preempt $P_1 \rightarrow [P_4, P_1]$. Run P_4 13-15 (finishes).
- $t = 15$: Queue $[P_1]$. Run P_1 15-16 (finishes).

Gantt: $P_1(0-2)$, $P_2(2-4)$, $P_1(4-6)$, $P_3(6-7)$, $P_2(7-9)$, $P_4(9-11)$, $P_1(11-13)$, $P_4(13-15)$, $P_1(15-16)$. Turnaround: $P_1 = 16$, $P_2 = 7$, $P_3 = 3$, $P_4 = 10$. Waiting: $P_1 = 9$, $P_2 = 3$, $P_3 = 2$, $P_4 = 6$. Avg Wait $= 20/4 =$ **5 ms**.

(c) Comparison: SRTF gives the better average waiting time (3 ms vs 5 ms). For P_3 (arrives 4), first response: SRTF starts at $t = 4$ (immediate), RR starts at $t = 6$. SRTF gives better first response for P_3 as well (0 ms wait vs 2 ms wait).

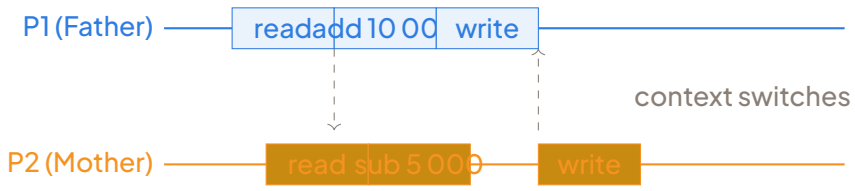
4 SECTION 4: PROCESS SYNCHRONISATION (LESSON 48)

In the previous section we saw how the scheduler decides which process runs next. But when several processes run concurrently and share data, the order in which their individual instructions interleave becomes critical. A tiny change in timing can turn a correct program into one that loses money, corrupts a file, or hangs forever. This section studies the techniques that guarantee correct cooperation: **process synchronisation**.

4.1 Why Synchronisation is Needed: Cooperating Processes and Interleaving

Processes are often independent, but many useful programs require cooperating processes that share data through shared memory, shared files, or message queues. Consider a mobile banking application used by a family in Douala: the father checks the balance while the mother transfers money to a sibling in Bafoussam. Both operations are carried out by processes that read and write the same `balance` variable in the bank's server memory.

The danger comes from the fact that a high-level statement such as `balance = balance + amount` is not executed in one indivisible step. It is compiled into at least three machine instructions: **load** the value of `balance` into a register, **add** the amount, **store** the result back. The scheduler may switch from one process to another between any two of these instructions.



Interleaving of the machine instructions of two cooperating processes sharing the variable `balance`. The scheduler switches between P1 and P2 between their read, modify and write steps.

The figure shows a possible interleaving:

1. P1 reads balance (50 000).
2. P2 reads balance (50 000).
3. P1 computes 50 000 + 10 000 = 60 000 and writes it.
4. P2 computes 50 000 - 5 000 = 45 000 and writes it.

The final balance is 45 000 FCFA — the father's deposit of 10 000 FCFA has been lost. This is a classic **race condition**.

DEFINITION **Race Condition**

A race condition occurs when multiple processes access and manipulate shared data concurrently, and the final result depends on the particular order in which the accesses take place.

WARNING **Lost Update — The Classic Trap**

The example above is called a lost update. It happens because the sequence `read` → `modify` → `write` is not atomic: the scheduler can switch processes between any two machine instructions. Never assume that a high-level statement like `balance = balance + amount` executes in one indivisible step.

4.2 The Critical Section Problem

To avoid race conditions, we must ensure that when one process is executing the sensitive part of its code that accesses shared data, no other process enters its own sensitive part for the same data. This sensitive part is called a **critical section**.

DEFINITION **Critical Section**

A critical section is a code segment in which a process accesses shared variables (or shared resources such as files or printers) and which must not be executed concurrently with another process's critical section for the same resource.

Each process can therefore be seen as a loop of four parts: an entry section (requesting permission to enter), the critical section, an exit section (releasing the resource), and a remainder section (all the other code). A correct solution to the critical-section problem must satisfy three requirements:

PROPERTY **Requirements of a Critical-Section Solution**

1. Mutual Exclusion: If process P_i is executing in its critical section, no other process P_j ($j \neq i$) may be in its critical section for the same resource.
2. Progress: If no process is currently in its critical section and some processes wish to enter theirs, then the choice of the next process to enter cannot be postponed indefinitely.
3. Bounded Waiting: There exists a bound on the number of times other processes are allowed to enter their critical sections after a process has requested entry and before

that request is granted. (No starvation.)

4.3 Software Approaches (Brief Overview)

Before hardware support existed, computer scientists devised pure software algorithms.

- **Strict Alternation:** A shared variable `turn` indicates whose turn it is. Process *i* busy-waits with `while (turn != i);`. This satisfies mutual exclusion but violates progress: it forces strict alternation even when the other process has no work to do.
- **Peterson's Solution (1981):** For two processes, it uses two boolean flags `interested[2]` and a variable `turn`. It satisfies all three requirements on paper, but relies on busy waiting and assumes that reads and writes to shared variables are atomic — an assumption that fails on modern multi-core processors without memory barriers.

WARNING **GCE Note**

Software solutions like Peterson's are not required in detail for the GCE examination. You should know they exist historically, but the examinable solutions are based on hardware atomic instructions and semaphores.

4.4 Hardware Support for Synchronisation

Modern processors provide atomic instructions that cannot be interrupted once started.

- **Disabling Interrupts:** On a single-core system, a process can disable interrupts before entering its critical section and re-enable them after leaving, which prevents any context switch. Drawbacks: it only works on uniprocessors, it gives too much power to user processes, and it blocks all I/O handling.
- **Test-and-Set (TSL) Instruction:** An atomic hardware instruction that reads a boolean variable and sets it to `true` in one indivisible operation:

```
boolean test_and_set(boolean *target) {
  boolean rv = *target;
  *target = true;
  return rv;
}
```

A spinlock using TSL then looks like this:

```
while (test_and_set(&lock)) ; // busy wait
// critical section
lock = false; // release
```

This works on multiprocessors but still uses busy waiting, which wastes CPU cycles while a process spins on the lock.

4.5 Semaphores: The Classic Synchronisation Tool

A **semaphore** is an integer variable that, apart from initialisation, is accessed only through two standard atomic operations: `wait()` (originally `P`, from the Dutch proberen — to test) and `signal()` (originally `V`, from verhogen — to increment). The atomicity of these operations is guaranteed by the operating system with hardware support.

DEFINITION **Semaphore**

A semaphore S is an integer variable that can only be accessed via two indivisible (atomic) operations:

`wait(S):` while ($S \leq 0$) wait; $S = S - 1$;

`signal(S):` $S = S + 1$;

In a good implementation, the waiting inside `wait` is done by blocking the process (putting it to sleep in a queue) rather than by busy waiting, so no CPU time is wasted.

- **Binary semaphore (mutex):** Takes only the values 0 or 1. Used for mutual exclusion.
- **Counting semaphore:** Takes any non-negative integer value. Used to control access to a pool of several identical resources (for example, 3 printers → semaphore initialised to 3).

PROPERTY **Atomicity of wait and signal**

The operations `wait` and `signal` are executed atomically: once started, they run to completion without interruption. This is the key property that makes semaphores a correct synchronisation primitive.

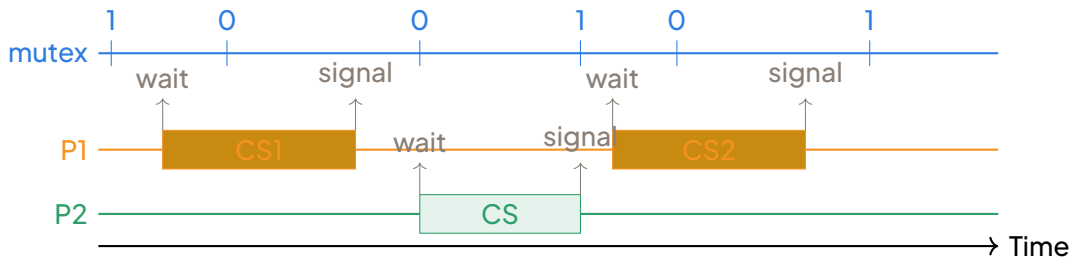
4.6 Protecting a Critical Section with a Binary Semaphore (Mutex)

The standard pattern for mutual exclusion uses a binary semaphore `mutex` initialised to 1.

METHOD **Protecting a Critical Section with a Mutex**

1. Declare a binary semaphore `mutex` and initialise it to `1`.
2. At the entry of the critical section, execute `wait(mutex)`.
3. Execute the critical section code.
4. At the exit of the critical section, execute `signal(mutex)`.

```
wait(mutex);
// critical section: access shared data
signal(mutex);
// remainder section
```



Timeline showing mutual exclusion enforced by a mutex semaphore. While P1 is in its critical section (CS1), mutex = 0, so P2 blocks at `wait()` until P1 executes `signal()`.

WARNING **Common Mistakes with Semaphores**

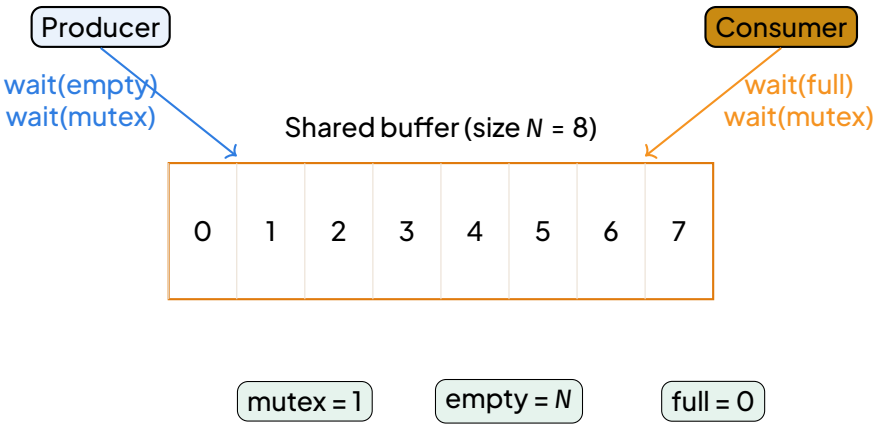
1. Wrong order: Executing `signal` before `wait` (or omitting one of them) breaks mutual exclusion or causes deadlock.
2. Forgetting `signal`: If a process crashes or returns early without signalling, the semaphore stays at 0 and all other processes block forever.
3. Initialisation errors: `mutex` must start at 1; `empty` at the buffer size N; `full` at 0. Wrong initial values cause immediate deadlock or buffer overflow.

4.7 Classical Problem: Producer–Consumer (Bounded Buffer)

A **producer** process generates data items and puts them into a shared buffer of fixed size N; a **consumer** process removes items from the buffer. The producer must not add an item when the buffer is full, the consumer must not remove an item when the buffer is empty, and the two must never access the buffer's internal pointers simultaneously.

We need three semaphores:

- `mutex` (binary, initialised to 1) — protects the buffer data structure itself.
- `empty` (counting, initialised to N) — counts empty slots; the producer waits on it.
- `full` (counting, initialised to 0) — counts filled slots; the consumer waits on it.



Bounded buffer with a producer, a consumer and the three semaphores. The producer waits on `empty` before inserting; the consumer waits on `full` before removing; `mutex` protects the buffer itself.

EXEMPLE **Producer–Consumer Pseudocode**

Shared declarations:

```
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
item buffer[N];
int in = 0, out = 0;
```

Producer:

```
while (true) {
  item next = produce_item();
  wait(empty);
  wait(mutex);
  buffer[in] = next;
  in = (in + 1) % N;
  signal(mutex);
  signal(full);
}
```

Consumer:

```
while (true) {
  wait(full);
  wait(mutex);
  item next = buffer[out];
  out = (out + 1) % N;
  signal(mutex);
  signal(empty);
  consume_item(next);
}
```

WARNING **Order of the wait() Calls**

The order `wait(empty)` then `wait(mutex)` in the producer (and `wait(full)` then `wait(mutex)` in the consumer) is crucial. If we reversed them (mutex first), a deadlock could occur: the producer holds `mutex`, finds the buffer full, and waits on `empty`; the consumer needs `mutex` to remove an item — a circular wait.

4.8 Other Classical Synchronisation Problems (Exam Awareness)

The GCE syllabus expects you to name and briefly describe these problems; you are not required to write full semaphore code for them.

- **Readers–Writers Problem:** Multiple readers may read a shared database simultaneously, but a writer must have exclusive access. Variants: readers' preference (a continuous stream of readers can starve writers) and writers' preference (writers can starve readers).
- **Dining Philosophers Problem:** Five philosophers sit around a table with five forks, one between each pair of plates. Each philosopher needs two forks to eat. The challenge is to design a protocol that avoids deadlock (for example, if all five pick up their left fork simultaneously, nobody can ever pick up the right one) and starvation.

4.9 Monitors: A Higher-Level Construct

Semaphores are low-level and error-prone (a forgotten `signal`, a wrong order of calls). A **monitor** is a higher-level synchronisation construct, supported by some programming languages or provided by the operating system, that encapsulates shared data together with the operations on it and guarantees that only one process can be active inside the monitor at any time.

DEFINITION **Monitor**

A monitor is a synchronisation construct that consists of:

- shared data (variables);
- a set of procedures (methods) that operate on that data;
- initialisation code.

Key property: mutual exclusion is automatic — only one process can execute a monitor procedure at a time. For waiting on conditions, monitors provide condition variables with the operations `wait()`, `signal()` and `broadcast()`.

DID YOU KNOW? **Monitors in Java**

In Java, every object has an intrinsic lock (a monitor). The `synchronized` keyword on a method or block provides the mutual exclusion, while `wait()`, `notify()` and `notifyAll()` on the object act as condition-variable operations. This is the practical realisation of the monitor concept that you will meet in programming.

4.10 Worked Example: Tracing a Producer–Consumer Execution

WORKED EXAMPLE 6 **Trace of a Bounded Buffer with N = 3**

Scenario: Buffer size $N = 3$. Initial state: `empty = 3`, `full = 0`, `mutex = 1`. The producer produces items A then B; the consumer then consumes one item. Show the semaphore values after each step.

SOLUTION

We trace the execution assuming the producer runs first, then the consumer.

Step	Process	Operation	empty	full	mutex	Buffer (in/out)
0	—	Initial	3	0	1	[, ,] (0/0)
1	P	wait(empty)	2	0	1	
2	P	wait(mutex)	2	0	0	
3	P	put A	2	0	0	[A, ,] (1/0)
4	P	signal(mutex)	2	0	1	
5	P	signal(full)	2	1	1	
6	P	wait(empty)	1	1	1	
7	P	wait(mutex)	1	1	0	
8	P	put B	1	1	0	[A, B,] (2/0)
9	P	signal(mutex)	1	1	1	
10	P	signal(full)	1	2	1	
11	C	wait(full)	1	1	1	
12	C	wait(mutex)	1	1	0	
13	C	take A	1	1	0	[, B,] (2/1)
14	C	signal(mutex)	1	1	1	
15	C	signal(empty)	2	1	1	

Final state: `empty = 2`, `full = 1`, `mutex = 1`; the buffer contains B at index 1, with `in = 2` and `out = 1`. Mutual exclusion held throughout: `mutex` never became negative and the two processes were never inside the buffer code at the same time.

4.11 Consolidation Exercises

EXERCISE 5 **Exercise 1 — Race Condition Identification**

Two processes share an integer `x` initially equal to 0. Process P1 executes `x = x + 1;` one thousand times in a loop; process P2 executes `x = x - 1;` one thousand times in a loop. Both run concurrently without any synchronisation.

1. What is the expected final value of `x` if every operation were atomic?
2. Explain why the actual final value may be different. Give a concrete interleaving of the machine instructions that produces a wrong result.

EXERCISE 6 **Exercise 2 — Semaphore Initialisation**

A system has 4 identical tape drives. Processes request a tape drive, use it, then release it.

1. What type of semaphore should be used, and with what initial value?
2. Write the `wait`/`signal` code for a process that needs one tape drive.

EXERCISE 7 **Exercise 3 — Producer–Consumer Deadlock**

In the bounded-buffer solution, suppose a programmer mistakenly writes the producer as:

```
wait(mutex);
wait(empty);
// ... put item ...
signal(mutex);
signal(full);
```

Explain how this can lead to deadlock. Illustrate with a scenario where $N = 1$.

EXERCISE 8 **Exercise 4 — Critical Section Requirements**

For each of the following scenarios, state which of the three critical-section requirements (mutual exclusion, progress, bounded waiting) is violated:

1. A process enters its critical section and never leaves (infinite loop inside it).
2. Two processes are in their critical sections simultaneously.
3. Process P1 wants to enter its critical section, but P2 repeatedly enters and leaves its own critical section, so P1 never gets in.

SOLUTION **Exercise 1**

1. Expected final value: 0, since the 1000 increments and the 1000 decrements cancel out.
2. The statement `x = x + 1` compiles into at least three machine instructions: `load x` into a register, `add 1`, `store` the register back to `x`. A losing interleaving:

(a) P1 loads x (value 0) into its register.

(b) P1 increments its register to 1.

(c) The scheduler switches to P2, which loads x (still 0, because P1 has not stored yet).

(d) P2 decrements its register to -1 and stores -1 into x.

(e) P1 resumes and stores 1 into x, overwriting P2's result.

One decrement has been lost. Repeated over many iterations, the final value can be far from 0.

SOLUTION **Exercise 2**

1. A counting semaphore, initialised to 4 (the number of available tape drives).
2. The code for one process:

```
wait(tape_drives); // acquire one drive
// use the tape drive
signal(tape_drives); // release it
```

SOLUTION **Exercise 3**

With $N = 1$, `empty` is initialised to 1.

1. Producer executes `wait(mutex)` → `mutex` = 0 (producer now holds the mutex).
2. Producer executes `wait(empty)` → `empty` = 0 (the slot is available, so it proceeds).
3. Producer puts the item, then `signal(mutex)` → `mutex` = 1, and `signal(full)` → `full` = 1.
4. The producer loops again: `wait(mutex)` → `mutex` = 0 (it holds the mutex again).
5. The producer executes `wait(empty)`: `empty` is 0, so the producer blocks inside wait(empty) while still holding the mutex.
6. The consumer runs: `wait(full)` succeeds (`full` goes from 1 to 0), then `wait(mutex)`: `mutex` is 0 (held by the blocked producer), so the consumer blocks too.
7. Both processes are blocked: the producer waits for `empty` (which only the consumer can signal), and the consumer waits for `mutex` (held by the producer). This is a deadlock.

SOLUTION **Exercise 4**

1. Progress is violated: no other process can ever enter because one process hogs the critical section indefinitely.
2. Mutual exclusion is violated.
3. Bounded waiting is violated: P1 waits indefinitely while P2 enters again and again (starvation of P1).

4.12 Key Points to Remember

KEY POINTS **Process Synchronisation — Summary for the GCE**

- Race condition: the result depends on the interleaving of instructions; classic example: the lost update on a shared bank balance.
- Critical section: code accessing a shared resource; a solution must guarantee mutual exclusion, progress and bounded waiting.
- Semaphore: an integer accessed only through the atomic operations `wait` (P) and `signal` (V). A binary semaphore (mutex) gives mutual exclusion; a counting semaphore manages a pool of resources.
- Mutex pattern: `wait(mutex)`; critical section; `signal(mutex)`; with `mutex = 1` initially.
- Producer–Consumer: three semaphores — `mutex = 1`, `empty = N`, `full = 0`. Always wait on the condition semaphore (`empty`/`full`) before waiting on `mutex`.
- Classical problems: Readers–Writers and Dining Philosophers — know their statements.
- Monitors: a high-level construct with automatic mutual exclusion and condition variables (Java `synchronized`).
- Common errors: wrong order of `wait`/`signal`, a missing `signal`, wrong initial values.

5 SECTION 5: DEADLOCK HANDLING AND INTEGRATION (LESSONS 49–50)

In Section 4 we saw that synchronisation mechanisms (semaphores, critical sections, monitors) allow processes to share resources safely. But synchronisation has a dark side: when several processes hold some resources and wait for others, the system can freeze completely.

This situation, called a **deadlock**, is one of the most feared problems in operating systems. In this final section we define deadlock precisely, model it with graphs, state the four conditions that make it possible, and study the four families of strategies to handle it: prevention, avoidance, detection and recovery — plus the surprising "do nothing" strategy. We close the chapter with an integration activity combining scheduling, synchronisation and deadlock analysis.

5.1 What is a Deadlock?

A concrete situation. Imagine the computer room of a school in Yaoundé with one printer and one scanner. Process P_1 (a student scanning a document) has grabbed the scanner and now requests the printer to print the scanned page. At the same time, process P_2 has grabbed the printer and now requests the scanner to scan the page it wants to print next. P_1 waits for the printer, held by P_2; P_2 waits for the scanner, held by P_1. Neither can ever proceed, and neither will ever release what it holds. The two processes are blocked forever.

DEFINITION **Deadlock**

A **deadlock** is a situation in which a set of processes is permanently blocked because each process in the set holds at least one resource and is waiting for a resource held by another process in the set. No process can proceed, release its resources, or be awakened without external intervention.

DID YOU KNOW? **The traffic jam analogy**

Deadlock also occurs on roads: at a crossroads in Douala, if four cars each enter the junction and each waits for the car on its right to move, all four are stuck forever — a real-world circular wait. Traffic lights are a prevention mechanism; a traffic policeman who orders one car to reverse is a recovery mechanism.

5.2 System Model and the Resource Allocation Graph

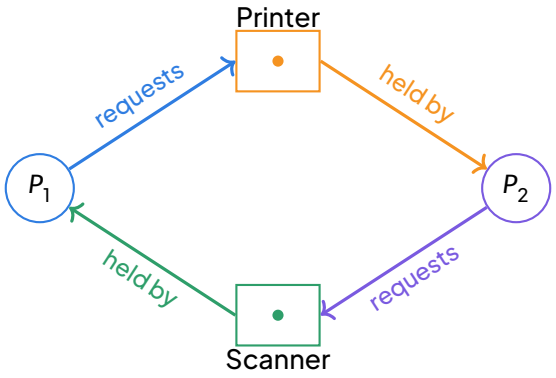
To reason rigorously about deadlocks we need a model. The system contains **resource types** (printer, scanner, memory region, file, semaphore…), each type having one or more identical **instances** (e.g. 2 printers). Every process uses a resource through a strict protocol:

1. **Request**: the process asks for the resource; if it is not free, the process waits.
2. **Use**: the process works with the resource.
3. **Release**: the process gives the resource back.

A deadlock can only arise inside this request–use–release cycle. The state of the system is pictured with a **Resource Allocation Graph** (RAG):

- a **circle** represents a process;
- a **rectangle** represents a resource type, with one **dot** per instance;

- a **request edge** $P_i \rightarrow R_j$ means P_i is waiting for an instance of R_j;
- an **assignment edge** $R_j \rightarrow P_i$ (drawn from a dot) means one instance of R_j is held by P_i.



RAG of the printer–scanner deadlock: the cycle P_1 → Printer → P_2 → Scanner → P_1 with one instance per resource means a certain deadlock.

METHOD **Analysing a Resource Allocation Graph**

1. Draw all processes, resources (with one dot per instance) and all request/assignment edges.
2. Look for a cycle in the graph.
3. If there is no cycle: no deadlock is possible.
4. If there is a cycle and every resource in the cycle has exactly one instance: deadlock is certain.
5. If there is a cycle but some resources have several instances: deadlock is only possible — check whether another process outside the cycle can release an instance and break it.

WARNING **A cycle does not always mean deadlock**

A very common exam mistake is to write "cycle ⇒ deadlock" without qualification. This is only true when each resource involved has one instance. With several instances, a cycle is a necessary but not sufficient condition: an instance held by a process outside the cycle may be released and unblock the cycle.

5.3 The Four Necessary Conditions (Coffman Conditions)

PROPERTY **Coffman necessary conditions for deadlock**

A deadlock can occur only if all four of the following conditions hold simultaneously:

1. Mutual exclusion: at least one resource is non-sharable (only one process can use it at a time).
2. Hold and wait: a process holds at least one resource while waiting for another.
3. No preemption: a resource cannot be taken away from a process by force; it is released only voluntarily.
4. Circular wait: there exists a cycle of processes $P_1, P_2, \ldots, P_n$ such that each waits for a resource held by the next (and P_n waits for P_1).

The four conditions are necessary, not individually sufficient: removing any one of them makes deadlock impossible. (The justification is direct from the definition and is examinable as a short essay question.)

5.4 Deadlock Prevention

Deadlock prevention consists in designing the system so that at least one Coffman condition can never hold:

- **Negate mutual exclusion**: make resources sharable where possible (e.g. spooled printer queue). Often impossible by nature of the device.
- **Negate hold and wait**: force each process to **request all its resources at once** before starting; it runs only when everything is available. Alternatively, a process must release everything before requesting a new resource.
- **Negate no preemption**: if a process holding resources is denied a new request, forcibly take its resources away (works for memory or CPU, not for a printer).
- **Negate circular wait**: impose a **total ordering** on resource types (printer = 1, scanner = 2, disk = 3…) and require every process to request resources in increasing order. Then a circular wait is impossible, because a cycle would require a resource number to be strictly smaller than itself.

WARNING **Prevention has a cost**

Breaking “hold and wait” by requesting all resources at once is safe but wasteful: a process that needs the printer only at the end of its execution keeps it idle from the start, so resource utilisation drops and starvation may appear. Every deadlock strategy trades safety against performance — a classic GCE discussion point.

5.5 Deadlock Avoidance: Safe States and the Banker's Algorithm

Prevention is rigid. **Avoidance** is more flexible: the system grants requests dynamically, but only when the request leaves the system in a **safe state**. Avoidance requires that each process declares its **maximum claim** (Max) for each resource type in advance.

DEFINITION **Safe and unsafe states**

A state is safe if there exists at least one ordering of the processes (a safe sequence) such that each process can obtain its remaining needs from the currently available resources plus the resources released by the processes that finish before it. A state that is not safe is unsafe. An unsafe state does not guarantee a deadlock, but a deadlock is only reachable from an unsafe state.

The **Banker's algorithm** (Dijkstra) implements avoidance. Like a careful banker, the OS never grants a loan (resource) that could make it impossible to satisfy all its clients. For each process it maintains:

- **Max** (claim): the maximum number of instances of each resource the process may need;
- **Allocation**: what the process currently holds;
- **Need** = Max − Allocation: what it may still request;
- **Available**: the free instances in the system (a single vector).

METHOD **Banker's safety algorithm**

1. Compute Need = Max − Allocation for every process.
2. Set Work = Available. Mark all processes "unfinished".
3. Find an unfinished process P_i with $\text{Need}_i \le \text{Work}$ (component by component). If none exists, go to step 5.
4. Pretend P_i runs to completion and releases everything: Work = Work + Allocation_i; mark P_i finished. Return to step 3.
5. If all processes are finished, the state is safe (the order found is a safe sequence); otherwise it is unsafe.

EXEMPLE **Safety check with 3 processes and 2 resources**

Consider resources A (10 instances) and B (7 instances):

	Allocation		Max		Need	
	A	*B*	*A*	*B*	*A*	*B*
P_1	2	1	5	3	3	2
P_2	3	2	4	4	1	2
P_3	2	1	6	3	4	2

Available = (10 − 7, 7 − 4) = (3, 3).

- Work = (3, 3): P_2 has Need (1, 2) ≤ (3, 3) ⇒ run P_2; Work = (3, 3) + (3, 2) = (6, 5).
- P_1 has Need (3, 2) ≤ (6, 5) ⇒ run P_1; Work = (8, 6).
- P_3 has Need (4, 2) ≤ (8, 6) ⇒ run P_3; Work = (10, 7).

All processes finish: the state is safe, with safe sequence $\langle P_2, P_1, P_3 \rangle$.

When a new request arrives, the OS pretends to grant it (updates Allocation, Need, Available), then runs the safety check: if the resulting state is safe, the request is granted; otherwise the process waits.

5.6 Deadlock Detection and Recovery

If the system neither prevents nor avoids deadlocks, it must **detect** them after the fact:

- **Single instance per resource**: periodically search for a **cycle** in the RAG (or in its simplified form, the wait-for graph where nodes are processes and an edge $P_i \rightarrow P_j$ means P_i waits for a resource held by P_j).
- **Multiple instances**: run a **detection algorithm** identical in spirit to the safety check, but using current requests instead of maximum needs; any process that can never be marked finished is deadlocked.

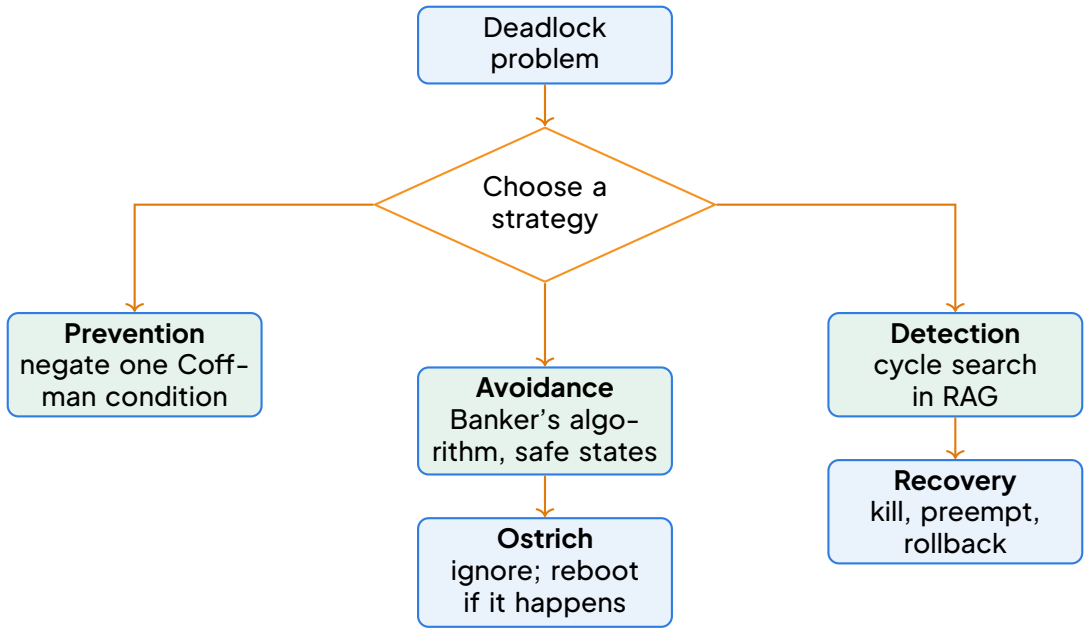
Once a deadlock is detected, **recovery** options are:

1. **Process termination**: kill one process at a time (or all deadlocked processes) until the cycle breaks — choose victims by priority, work already done, resources held.
2. **Resource preemption**: forcibly take resources from a victim and give them to others; the victim is **rolled back** to a previous checkpoint and restarted later.

DID YOU KNOW? **The ostrich algorithm**

Many real operating systems (Windows, Linux) simply ignore deadlocks — the so-called ostrich algorithm. Deadlocks are rare, and prevention/avoidance mechanisms slow the system down permanently to guard against an event that may never happen. Engineers therefore accept a rare freeze (fixed by a reboot) rather than pay a constant performance

cost. It is a deliberate engineering trade-off, not an oversight.



The four families of deadlock handling strategies.

5.7 Integration Activity (Lesson 50): A Complete Case Study

WORKED EXAMPLE 7 **Scheduling, synchronisation and deadlock combined**

A school's server runs three processes. P_1 prints report cards (needs the printer, then the scanner), P_2 scans archives (needs the scanner, then the printer), P_3 computes statistics (CPU only). Burst times: P_1 = 6 ms, P_2 = 4 ms, P_3 = 2 ms; all arrive at time 0 in the order P_1, P_2, P_3.

1. Compute average waiting time under FCFS and under SJF (non-preemptive).
2. The printer and scanner are protected by semaphores `S1` and `S2`. P_1 executes `wait(S1)` then `wait(S2)`; P_2 executes `wait(S2)` then `wait(S1)`. What can happen?
3. Propose a prevention solution and justify it with the Coffman conditions.

SOLUTION

1. Scheduling. FCFS order: P_1, P_2, P_3. Waiting times: P_1 = 0, P_2 = 6, P_3 = 6 + 4 = 10; average = (0 + 6 + 10)/3 ≈ 5.33 ms. SJF order: $P_3(2), P_2(4), P_1(6)$. Waiting times: P_3 = 0, P_2 = 2,

P_1 = 2 + 4 = 6; average = (0 + 2 + 6)/3 ≈ 2.67 ms. SJF is better, as expected.
2. Synchronisation. If P_1 acquires `S1` (printer) and is then preempted, and P_2 acquires `S2` (scanner), each process blocks forever on its second `wait`: this is exactly the printer–scanner deadlock of the RAG above. The four Coffman conditions hold: mutual exclusion (one printer, one scanner), hold and wait, no preemption of semaphores, circular wait.
3. Prevention. Impose a resource ordering: every process must request `S1` before `S2`. Then P_2 must also lock the printer first; whichever process gets the printer first can complete, and the circular wait condition is destroyed, so deadlock becomes impossible. (Equivalently, both processes could request both semaphores atomically, breaking hold and wait — at the cost of lower utilisation.)

EXERCISE 9 **Deadlock analysis**

A system has one resource type with 4 instances and three processes with Max = (3, 3, 2) and Allocation = (1, 2, 1).

1. Compute the Need matrix and the Available vector.
2. Is the state safe? If yes, give a safe sequence.
3. P_1 now requests 1 more instance. Should the request be granted according to the Banker's algorithm?

SOLUTION **Exercise 1**

1. Need = Max - Allocation = (3 - 1, 3 - 2, 2 - 1) = (2, 1, 1). Available = 4 - (1 + 2 + 1) = 0. 2. Work = 0: no process has Need ≤ 0 (all needs are positive), so no process can be guaranteed to finish. The state is unsafe (not yet a deadlock, but one bad request away). 3. Pretend to grant: Allocation becomes (2, 2, 1), Need becomes (1, 1, 1), Available becomes -1 — impossible; the request exceeds Available, so it cannot even be pretended. The request must be refused / delayed; granting it would be invalid, and the system is already unsafe.

KEY POINTS **Key points of Section 5**

- A **deadlock** is a permanent mutual blocking of processes, each holding resources and waiting for others.
- Deadlock requires all four Coffman conditions: mutual exclusion, hold and wait, no preemption, circular wait.
- In a RAG, no cycle ⇒ no deadlock; a cycle with one instance per resource ⇒ certain

deadlock; with several instances, a cycle is only a warning.

- Prevention negates one condition (request-all-at-once, resource ordering); avoidance keeps the system in safe states (Banker's algorithm: Need = Max - Allocation, find Need ≤ Work, release, repeat); detection finds cycles after the fact, with recovery by killing, preemption or rollback; the ostrich algorithm deliberately does nothing.
- Every strategy trades safety against resource utilisation and performance.

6 INTEGRATION ACTIVITY

6.1 Aims of the activity

This integration activity mobilises the core concepts of process management covered in Lessons 44–49: process scheduling (both non-preemptive and preemptive), process synchronisation using semaphores, and deadlock handling. By working on a single realistic scenario — a cyber-café server in Yaoundé — you will:

- Compute and compare scheduling metrics (waiting time, turnaround time) for FCFS, SJF and Round Robin, and justify a policy choice for customer satisfaction.
- Design a synchronisation mechanism (semaphore) to protect a shared printer and explain the race condition that occurs without it.
- Model a deadlock situation with a Resource Allocation Graph, verify the four necessary conditions, and propose both a prevention and a recovery strategy.
- Present your findings as a professional recommendation to the café owner, demonstrating the ability to integrate theory into practice — a key skill for the GCE Advanced Level examination.

6.2 Situation

Yaoundé Cyber-Café "Digital Hub" operates a single server that handles customer jobs. At 08:00 the server receives five jobs with the following characteristics (arrival time in minutes after 08:00, burst time in minutes):

Job ID	Description	Arrival Time	Burst Time
J1	Printing a 20-page document	0	8
J2	Scanning a batch of photos	1	4
J3	Browsing session (video streaming)	2	9
J4	Video rendering (short clip)	3	5
J5	File download (large dataset)	4	2

The café has only **one printer** and **one scanner**, both connected to the server. Print jobs are spooled to the printer; scan jobs are sent to the scanner. The server runs a simple job scheduler. The owner wants to minimise average waiting time and turnaround time to keep customers happy, but also needs to ensure that print jobs never interleave (which would produce garbled pages) and that the system never deadlocks when customers simultaneously request the printer and scanner.

Later that morning, two customers (C1 and C2) arrive almost simultaneously:

- C1 starts a print job (acquires the printer) and then requests the scanner to scan a receipt.
- C2 starts a scan job (acquires the scanner) and then requests the printer to print a confirmation.

Both requests are made while the other resource is still held.

6.3 Tasks

1. **Scheduling analysis**
   (a) Draw the Gantt chart for each of the following scheduling policies: First-Come-First-Served (FCFS), Shortest Job First (SJF) non-preemptive, and Round Robin with time quantum $q = 3$ minutes.
   (b) For each policy, compute the waiting time and turnaround time of every job, then the average waiting time and average turnaround time.
   (c) Based on the computed averages and the nature of the jobs (interactive vs. batch), recommend the best scheduling policy for customer satisfaction. Justify your choice.
2. **Synchronisation of the printer**
   (a) Write clear pseudocode (using `wait(S)` and `signal(S)` operations on a binary semaphore) that protects the critical section where a print job sends data to the printer.
   (b) Explain, in terms of a race condition, what would happen if two print jobs entered the critical section concurrently (describe the garbled output).
3. **Deadlock analysis**
   (a) Draw the Resource Allocation Graph (RAG) for the situation where C1 holds the printer and requests the scanner, while C2 holds the scanner and requests the printer.
   (b) Verify that all four necessary conditions for deadlock (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait) are satisfied.
   (c) Propose **one deadlock prevention** technique (choose from: eliminating Hold and Wait, allowing Preemption, or imposing a Resource Ordering) and explain how it would be applied in this café.
   (d) Propose **one deadlock recovery** technique (choose from: Process Termination or Resource Preemption) and describe the steps the server would take to recover.
4. **Presentation** Prepare a 3–minute oral summary (as if advising the café owner) that integrates your scheduling recommendation, synchronisation solution, and deadlock strategy.

METHOD **How to compute scheduling metrics**

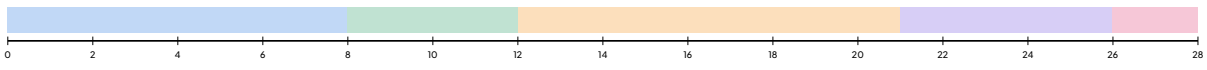
For every job, once the Gantt chart is drawn:

1. Read the **completion time** CT: the instant at which the job finishes.
2. Compute the **turnaround time**: $TT = CT$ − arrival time.
3. Compute the **waiting time**: $WT = TT$ − burst time (total time spent in the ready queue).
4. Average over all jobs: $\overline{WT} = \frac{\sum WT_i}{n}$ and $\overline{TT} = \frac{\sum TT_i}{n}$.

6.4 Model answer

1. Scheduling analysis

FCFS (First-Come-First-Served). Jobs are served strictly in order of arrival: J1, J2, J3, J4, J5. Since J1 arrives at time 0 and runs for 8 minutes, all other jobs queue behind it.



Gantt chart for FCFS

Completion times: $CT_{J1} = 8$, $CT_{J2} = 12$, $CT_{J3} = 21$, $CT_{J4} = 26$, $CT_{J5} = 28$.

Turnaround times ($TT = CT$ − arrival):

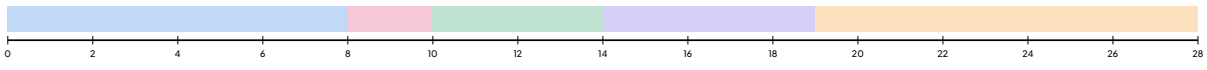
$$TT_{J1} = 8 - 0 = 8, \qquad TT_{J2} = 12 - 1 = 11, \qquad TT_{J3} = 21 - 2 = 19,$$
$$TT_{J4} = 26 - 3 = 23, \qquad TT_{J5} = 28 - 4 = 24.$$
$$\text{Average } TT = \frac{8 + 11 + 19 + 23 + 24}{5} = \frac{85}{5} = 17.0 \text{ min.}$$

Waiting times ($WT = TT$ − burst):

$$WT_{J1} = 8 - 8 = 0, \qquad WT_{J2} = 11 - 4 = 7, \qquad WT_{J3} = 19 - 9 = 10,$$
$$WT_{J4} = 23 - 5 = 18, \qquad WT_{J5} = 24 - 2 = 22.$$
$$\text{Average } WT = \frac{0 + 7 + 10 + 18 + 22}{5} = \frac{57}{5} = 11.4 \text{ min.}$$

SJF (Non-preemptive). At each decision point, choose among the arrived jobs the one with the shortest burst time; once started, a job runs to completion.

- Time 0: only J1 has arrived ⇒ J1 runs 0–8.
- Time 8: arrived jobs are J2 (4), J3 (9), J4 (5), J5 (2). Shortest is J5 ⇒ runs 8–10.
- Time 10: remaining J2 (4), J4 (5), J3 (9). Shortest is J2 ⇒ runs 10–14.
- Time 14: remaining J4 (5), J3 (9). Shortest is J4 ⇒ runs 14–19.
- Time 19: only J3 remains ⇒ runs 19–28.



Gantt chart for SJF (non-preemptive)

Completion times: $CT_{J1} = 8$, $CT_{J5} = 10$, $CT_{J2} = 14$, $CT_{J4} = 19$, $CT_{J3} = 28$.

Turnaround times:

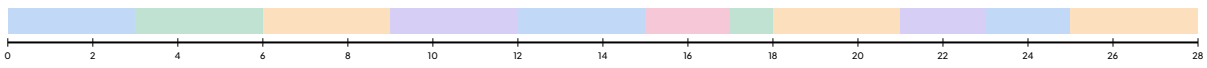
$$TT_{J1} = 8 - 0 = 8, \quad TT_{J2} = 14 - 1 = 13, \quad TT_{J3} = 28 - 2 = 26,$$
$$TT_{J4} = 19 - 3 = 16, \quad TT_{J5} = 10 - 4 = 6.$$
$$\text{Average } TT = \frac{8 + 13 + 26 + 16 + 6}{5} = \frac{69}{5} = 13.8 \text{ min}.$$

Waiting times:

$$WT_{J1} = 0, \quad WT_{J2} = 13 - 4 = 9, \quad WT_{J3} = 26 - 9 = 17,$$
$$WT_{J4} = 16 - 5 = 11, \quad WT_{J5} = 6 - 2 = 4.$$
$$\text{Average } WT = \frac{0 + 9 + 17 + 11 + 4}{5} = \frac{41}{5} = 8.2 \text{ min}.$$

Round Robin (q = 3). The ready queue is managed in arrival order (FCFS); each job receives at most 3 minutes of CPU per turn, then goes to the back of the queue if not finished. New arrivals join the tail of the queue. Careful trace:

- Time 0: J1 runs 0–3 (remaining 5). Queue: J2, J3, J4, J1. (J5 arrives at time 4 and joins the tail.)
- Time 3: J2 runs 3–6 (remaining 1). Queue: J3, J4, J1, J5, J2.
- Time 6: J3 runs 6–9 (remaining 6). Queue: J4, J1, J5, J2, J3.
- Time 9: J4 runs 9–12 (remaining 2). Queue: J1, J5, J2, J3, J4.
- Time 12: J1 runs 12–15 (remaining 2). Queue: J5, J2, J3, J4, J1.
- Time 15: J5 runs 15–17 (burst 2 < q, finishes). Queue: J2, J3, J4, J1.
- Time 17: J2 runs 17–18 (remaining 1, finishes). Queue: J3, J4, J1.
- Time 18: J3 runs 18–21 (remaining 3). Queue: J4, J1, J3.
- Time 21: J4 runs 21–23 (remaining 2, finishes). Queue: J1, J3.
- Time 23: J1 runs 23–25 (remaining 2, finishes). Queue: J3.
- Time 25: J3 runs 25–28 (finishes).



Gantt chart for Round Robin (q = 3)

Completion times: $CT_{J1} = 25$, $CT_{J2} = 18$, $CT_{J3} = 28$, $CT_{J4} = 23$, $CT_{J5} = 17$.

Turnaround times:

$$TT_{J1} = 25 - 0 = 25, \quad TT_{J2} = 18 - 1 = 17, \quad TT_{J3} = 28 - 2 = 26,$$
$$TT_{J4} = 23 - 3 = 20, \quad TT_{J5} = 17 - 4 = 13.$$
$$\text{Average } TT = \frac{25 + 17 + 26 + 20 + 13}{5} = \frac{101}{5} = 20.2 \text{ min}.$$

Waiting times:

$$WT_{J1} = 25 - 8 = 17, \quad WT_{J2} = 17 - 4 = 13, \quad WT_{J3} = 26 - 9 = 17,$$
$$WT_{J4} = 20 - 5 = 15, \quad WT_{J5} = 13 - 2 = 11.$$
$$\text{Average } WT = \frac{17 + 13 + 17 + 15 + 11}{5} = \frac{73}{5} = 14.6 \text{ min.}$$

WARNING **Common mistakes in Round Robin traces**

- Do not forget a job that arrives while another is running: J5 arrives at time 4 and must join the tail of the queue.
- A job whose remaining burst is smaller than the quantum finishes before the quantum expires (J5 runs only 2 minutes, J2 only 1 minute at the end).
- A new arrival goes to the tail of the ready queue, never in front of jobs already waiting.

Comparison and recommendation.

Policy	Avg Waiting Time (min)	Avg Turnaround Time (min)
FCFS	11.4	17.0
SJF (non-preemptive)	8.2	13.8
Round Robin (q = 3)	14.6	20.2

SJF gives the lowest average waiting time (8.2 min) and average turnaround time (13.8 min) — this is expected, since SJF is provably optimal for average waiting time among non-preemptive policies. However, SJF can cause **starvation** of long jobs (J3 waits 17 minutes) and requires knowing burst times in advance. Round Robin guarantees fairness and good response time for interactive jobs (browsing, scanning), but its averages are worse here because of frequent context switches. FCFS is simple but penalises short jobs stuck behind long ones (the convoy effect: J5 waits 22 minutes behind J1).

Recommendation: since the owner's stated goal is to minimise average waiting and turnaround times, **SJF (non-preemptive)** is the best single policy for the batch-like jobs (printing, rendering, downloads). In practice, a hybrid is ideal: SJF for batch jobs and Round Robin with a small quantum for interactive sessions (browsing, scanning), so that interactive customers never feel the server is frozen. The owner should simply be warned that pure SJF may starve very long jobs.

2. Synchronisation of the printer

Semaphore pseudocode. A binary semaphore `printerSem`, initialised to 1 (printer free), guarantees mutual exclusion on the printer:

- `var printerSem : semaphore := 1` (1 = printer free, 0 = printer busy)
- **Procedure** `PrintJob(jobData)`:
  1. `wait(printerSem)` (acquire the printer; block if busy)

2. `sendToPrinter(jobData)` (critical section: one job at a time)
3. `signal(printerSem)` (release the printer; wake a waiting job)

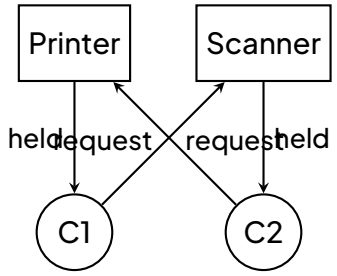
Because `wait` and `signal` are atomic (indivisible) operations, at most one process can be inside the critical section at any instant.

Race condition without the semaphore. If two print jobs (e.g., J1 and J4) enter `sendToPrinter` concurrently, their data streams interleave in the printer's buffer: a fragment of J1's page 1, then a fragment of J4's page 1, then J1 again, and so on. The printed pages are garbled — mixed text and images from both documents — and unusable. This is a classic **race condition**: the final result depends on the unpredictable relative timing of the processes accessing the shared resource without mutual exclusion. The semaphore removes the race by serialising access.

3. Deadlock analysis

Resource Allocation Graph (RAG). Circles represent processes, rectangles represent resources. An assignment edge (resource → process) means the resource is held by the process; a request edge (process → resource) means the process is waiting for the resource.

- Assignment edges: Printer → C1, Scanner → C2.
- Request edges: C1 → Scanner, C2 → Printer.



Resource Allocation Graph: the cycle C1 → Scanner → C2 → Printer → C1 reveals the deadlock

The graph contains a cycle: C1 → Scanner → C2 → Printer → C1. Since each resource has a single instance, a cycle in the RAG is a sufficient condition for deadlock: neither process can ever proceed.

Verification of the four necessary conditions.

1. **Mutual Exclusion:** the printer and the scanner are non-shareable; only one job can use each at a time. Satisfied.
2. **Hold and Wait:** C1 holds the printer while waiting for the scanner; C2 holds the scanner while waiting for the printer. Satisfied.
3. **No Preemption:** resources cannot be forcibly taken away; they are released only voluntarily by the holder. Satisfied.
4. **Circular Wait:** C1 waits for the scanner held by C2, and C2 waits for the printer held by C1 — a circular chain exists. Satisfied.

All four conditions hold simultaneously, so the system is deadlocked.

Deadlock prevention: Resource Ordering. Impose a global ordering on resources, e.g., Printer < Scanner, and require every process to request resources in increasing order only. In the café, any job needing both devices must request the printer first, then the scanner. C1 (print then scan) already follows the order; C2 would be forced to request the printer first as well. Then a circular wait becomes impossible: a cycle would require a process to hold a high-numbered resource while waiting for a lower-numbered one, which the rule forbids. The Circular Wait condition is broken, so deadlock can never occur.

Deadlock recovery: Process Termination. The server detects the deadlock (e.g., via a timeout on blocked jobs or a periodic cycle-detection algorithm on the RAG). It then **terminates one of the deadlocked processes** — say C2, the scan job. The scanner is released and allocated to C1, which completes its work and releases both resources. C2's job is restarted later (the customer re-submits the scan request). The system recovers at the cost of losing C2's progress; the victim is usually chosen to minimise cost (youngest job, least work done, lowest priority).

4. Presentation outline (3 minutes)

- **Opening (30 s):** "Good morning. I have analysed the server workload of Digital Hub. Here are my recommendations to maximise customer satisfaction and system reliability."
- **Scheduling (60 s):** show the comparison table. "SJF gives the lowest average waiting time (8.2 min) and turnaround time (13.8 min). For your mix of jobs, I recommend SJF for background jobs such as printing and downloads, combined with Round Robin (q = 3) for interactive browsing and scanning sessions. This keeps averages low while guaranteeing responsiveness."
- **Printer synchronisation (45 s):** "Without protection, two simultaneous print jobs interleave and produce garbled pages. A binary semaphore around the printer driver enforces mutual exclusion — a simple, proven solution with negligible overhead."
- **Deadlock strategy (45 s):** "The Resource Allocation Graph shows a cycle: all four deadlock conditions hold. Prevention by resource ordering — always request the printer before the scanner — eliminates circular wait entirely. Should a deadlock ever occur, the server will terminate the younger job and restart it, minimising disruption."
- **Closing (15 s):** "These three measures will make Digital Hub faster, fairer and crash-free. Thank you."

KEY POINTS **Key takeaways for the exam**

- Always draw the Gantt chart first, then compute $TT = CT$ − arrival and $WT = TT$ − burst; show the chart in your answer.
- SJF minimises average waiting time but can starve long processes; Round Robin guarantees fairness and good response time at the cost of more context switches.
- A binary semaphore (mutex) with atomic `wait`/`signal` operations solves the critical-section problem for a single shared resource.
- Deadlock requires all four conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait); prevention attacks one condition, recovery breaks the deadlock

after it occurs.

- In an integration question, link every answer back to the scenario (the café owner's goals) and justify each recommendation with computed values.

7 METHODS AND WORKED EXAMPLES

7.1 Method 1 — Drawing a Gantt Chart and Computing Performance Criteria

METHOD **Scheduling a Set of Processes with a Gantt Chart**

1. Read the data table carefully. For each process note its **arrival time** (AT) and **burst time** (BT, the CPU time it needs). Never confuse the two.
2. Identify the algorithm (FCFS, SJF, SRTF, Round Robin, Priority…) and whether it is **pre-emptive** or not.
3. Build the ready queue step by step. At each decision instant (an arrival, or the end of a burst/quantum), list the processes present and apply the algorithm's rule to choose the next one.
4. Draw the Gantt chart: a horizontal time axis, one block per execution interval, labelled with the process name, with the start and end times written below.
5. Compute for each process:
   - **Completion time** CT = time at which the process finishes;
   - **Turnaround time** TAT = CT − AT;
   - **Waiting time** WT = TAT − BT;
   - **Response time** RT = (first time the process gets the CPU) − AT.
6. Average the results over all processes. Give answers as fractions or decimals with a point (e.g. 8.25 ms).
7. Check: WT = TAT − BT must never be negative; the total length of the Gantt chart must equal the sum of burst times (if the CPU is never idle).

WARNING **Standard Assumptions in GCE Scheduling Problems**

Unless the question says otherwise, assume that: the cost of a **context switch** is negligible (0 ms); a process arriving at time t is available for scheduling at time t; and all times are integers. State these assumptions briefly if you use them.

WORKED EXAMPLE 8 **FCFS Scheduling — Cyber Café in Buea**

The manager of a cyber café in Buea observes four print jobs arriving at the single printer server. Modelled as CPU processes, they are:

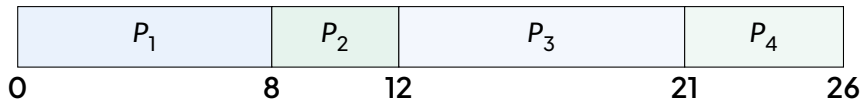
Process	Arrival time (ms)	Burst time (ms)
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Using First-Come First-Served (FCFS), draw the Gantt chart and compute the average turnaround time and average waiting time.

SOLUTION

Step 1 — Order of execution. FCFS serves processes in arrival order: P_1 (arrives at 0), then P_2 (1), then P_3 (2), then P_4 (3). Since FCFS is non-preemptive, each process runs to completion once started.

Step 2 — Gantt chart.



Gantt chart for FCFS.

P_1 runs from 0 to 8; P_2 from 8 to 12; P_3 from 12 to 21; P_4 from 21 to 26.

Step 3 — Criteria table. Recall: TAT = CT − AT and WT = TAT − BT.

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P_1	0	8	8	8 − 0 = 8	8 − 8 = 0
P_2	1	4	12	12 − 1 = 11	11 − 4 = 7
P_3	2	9	21	21 − 2 = 19	19 − 9 = 10
P_4	3	5	26	26 − 3 = 23	23 − 5 = 18

Step 4 — Averages.

$$\text{Average TAT} = \frac{8 + 11 + 19 + 23}{4} = \frac{61}{4} = 15.25 \text{ ms}$$

$$\text{Average WT} = \frac{0 + 7 + 10 + 18}{4} = \frac{35}{4} = 8.75 \text{ ms}$$

Conclusion. With FCFS, the average turnaround time is 15.25 ms and the average waiting time is 8.75 ms. Note how the long job P_3 penalises P_4: this is the **convoy effect**, a classic weakness of FCFS.

7.2 Method 2 — Applying Non-preemptive SJF

METHOD **Shortest Job First (Non-preemptive)**

1. At the moment the CPU becomes free, look only at the processes that have already arrived.
2. Choose the one with the smallest burst time. In case of a tie, use the arrival order (FCFS tie-break).
3. Once started, a process runs to completion (non-preemptive): do not interrupt it even if a shorter job arrives meanwhile.
4. If the CPU is free but no process has arrived yet, leave it idle until the next arrival.
5. Draw the Gantt chart, then compute CT, TAT, WT as in Method 1.
6. Reflex: SJF minimises the average waiting time among non-preemptive algorithms — a result the examiner may ask you to state.

WORKED EXAMPLE 9 **SJF versus FCFS**

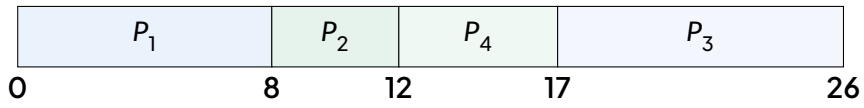
Using the same data as the previous exercise:

Process	Arrival time (ms)	Burst time (ms)
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Apply non-preemptive Shortest Job First, draw the Gantt chart and compute the average waiting time. Compare with FCFS.

SOLUTION

Step 1 — Decision at time 0. Only P_1 has arrived, so P_1 runs from 0 to 8 (non-preemptive: it cannot be interrupted).
Step 2 — Decision at time 8. P_2 (BT 4), P_3 (BT 9) and P_4 (BT 5) are all waiting. The shortest is P_2 (4 ms): it runs from 8 to 12.
Step 3 — Decision at time 12. Remaining: P_3 (9) and P_4 (5). The shortest is P_4: it runs from 12 to 17.
Step 4 — Decision at time 17. Only P_3 remains: it runs from 17 to 26.



Gantt chart for non-preemptive SJF.

Step 5 — Criteria table.

Process	AT	BT	CT	TAT	WT
P_1	0	8	8	8	0
P_2	1	4	12	11	7
P_4	3	5	17	14	9
P_3	2	9	26	24	15

$$\text{Average WT} = \frac{0 + 7 + 9 + 15}{4} = \frac{31}{4} = 7.75 \text{ ms}$$

Comparison. SJF gives an average waiting time of 7.75 ms against 8.75 ms for FCFS: SJF is better, as expected, since it is provably optimal for average waiting time. However, the long process P_3 is served last: SJF can cause **starvation** of long jobs.

7.3 Method 3 — Preemptive Scheduling: SRTF

METHOD **Shortest Remaining Time First**

1. SRTF is the preemptive version of SJF: the CPU always goes to the process with the smallest **remaining** burst time.
2. Re-examine the situation at every arrival: if the new process's burst time is smaller than the remaining time of the running process, preempt the running process and put it back in the ready queue with its updated remaining time.
3. Keep a small table of remaining times, updated at each event (arrival or completion).
4. On ties in remaining time, prefer the process that arrived first (or keep the running one to avoid useless context switches).
5. When computing the response time, remember it is measured at the first allocation of the CPU, which may be long before completion.

WORKED EXAMPLE 10 **SRTF with Preemption**

Consider the following processes:

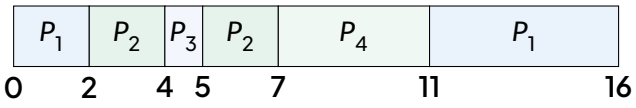
Process	Arrival time (ms)	Burst time (ms)
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

Schedule them with SRTF, draw the Gantt chart and compute the average turnaround time.

SOLUTION

Step 1 — Trace the events.

- t = 0: only P_1 is present; it starts. Remaining: P_1 = 7.
- t = 2: P_2 arrives (BT 4). P_1 has run 2 ms, so its remaining time is 7 − 2 = 5. Since 4 < 5, P_2 preempts P_1.
- t = 4: P_3 arrives (BT 1). P_2 has remaining 4 − 2 = 2. Since 1 < 2, P_3 preempts P_2.
- t = 5: P_3 finishes (it needed only 1 ms). P_4 arrives (BT 4). Ready queue: P_2 (remaining 2), P_1 (remaining 5), P_4 (4). The smallest is P_2: it resumes.
- t = 7: P_2 finishes. Remaining: P_4 (4) and P_1 (5). P_4 runs from 7 to 11.
- t = 11: P_4 finishes. P_1 resumes with 5 ms remaining, from 11 to 16.



Gantt chart for SRTF: note the two preemptions.

Step 2 — Criteria table.

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P_1	0	7	16	16	9
P_2	2	4	7	5	1
P_3	4	1	5	1	0
P_4	5	4	11	6	2

$$\text{Average TAT} = \frac{16 + 5 + 1 + 6}{4} = \frac{28}{4} = 7 \text{ ms}$$

Conclusion. SRTF gives an average turnaround time of 7 ms. (For comparison, non-preemptive SJF on the same data gives an average TAT of 8 ms: P_1 runs 0–7, then P_3 7–8, P_2 8–12, P_4 12–16, so TAT = 7, 10, 4, 11.) The gain comes at the price of two context switches. Long processes like P_1 may be delayed repeatedly: the starvation risk is even stronger than with SJF.

7.4 Method 4 — Round Robin Scheduling

METHOD **Round Robin with Quantum q**

1. Maintain a FIFO ready queue. New arrivals are added at the tail of the queue.
2. Give the CPU to the process at the head for at most **quantum** q time units.
3. If the process finishes before the quantum expires, remove it and take the next one immediately.
4. If the quantum expires first, preempt the process and put it back at the tail of the queue (after any process that arrived during the quantum — apply the convention stated in the question; if none is stated, add new arrivals first, then the preempted process).
5. Repeat until all processes complete. A process's remaining time decreases by q each time it uses a full quantum.
6. Reflex: a very large q makes RR behave like FCFS; a very small q wastes time in context switches.

WORKED EXAMPLE 11 **Round Robin with $q = 3$ ms**

Three processes arrive at a single-CPU system:

Process	Arrival time (ms)	Burst time (ms)
P_1	0	5
P_2	1	9
P_3	2	3

Apply Round Robin with quantum $q = 3$ ms. Draw the Gantt chart and compute the average waiting time and the response time of each process.

SOLUTION

Step 1 — Simulate the ready queue. Convention: when a quantum expires, newly arrived processes join the queue before the preempted process.

- $t = 0$: queue = $[P_1]$. P_1 runs 0–3 (remaining 2). During this quantum P_2 (at 1) and P_3 (at 2) arrive. Queue = $[P_2, P_3, P_1]$.
- $t = 3$: P_2 runs 3–6 (remaining 6). Queue = $[P_3, P_1, P_2]$.
- $t = 6$: P_3 runs 6–9; it needs exactly 3, so it finishes at 9. Queue = $[P_1, P_2]$.
- $t = 9$: P_1 runs 9–11; it needs only 2, so it finishes at 11. Queue = $[P_2]$.
- $t = 11$: P_2 runs 11–14 (remaining 3). Queue = $[P_2]$.
- $t = 14$: P_2 runs 14–17 and finishes.



Gantt chart for Round Robin, $q = 3$ ms.

Step 2 — Criteria table. Response time RT = first CPU allocation − AT.

Process	AT	BT	CT	TAT	WT	RT
P_1	0	5	11	11	6	0
P_2	1	9	17	16	7	2
P_3	2	3	9	7	4	4

For example, for P_1: WT = TAT − BT = 11 − 5 = 6 ms (it waited from 3 to 9, i.e. 6 ms — consistent). For P_2: RT = 3 − 1 = 2 ms (first run starts at $t = 3$); for P_3: RT = 6 − 2 = 4 ms.

$$\text{Average WT} = \frac{6 + 7 + 4}{3} = \frac{17}{3} \approx 5.67 \text{ ms}$$

Conclusion. Round Robin guarantees every process a share of the CPU within a bounded delay (good response times, no starvation), which is why it is the basis of time-sharing systems. The cost is more context switches than FCFS or SJF.

7.5 Method 5 — Priority Scheduling

METHOD **Priority Scheduling (Preemptive or Not)**

1. Check the convention: in most GCE problems, the smallest number means the highest priority — but always read the statement.
2. If non-preemptive: when the CPU is free, pick the waiting process with the highest priority; let it finish.
3. If preemptive: at every arrival, compare priorities; a newly arrived process with a higher priority takes the CPU immediately.
4. Break ties with FCFS (earliest arrival first).
5. Mention the danger: **starvation** of low-priority processes, and its remedy, **ageing** (gradually raising the priority of processes that wait long).

WORKED EXAMPLE 12 **Preemptive Priority Scheduling**

A hospital management system in Yaoundé handles four tasks (1 = highest priority):

Process	Arrival (ms)	Burst (ms)	Priority
P_1	0	10	3
P_2	1	1	1
P_3	2	2	4
P_4	3	1	2

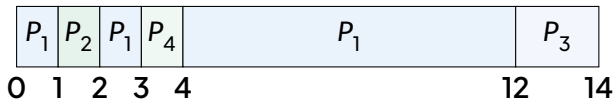
Apply preemptive priority scheduling, draw the Gantt chart and compute the average turnaround time.

SOLUTION

Step 1 — Trace the events.

- t = 0: only P_1 (priority 3) is present; it starts.
- t = 1: P_2 arrives with priority 1 (highest). It preempts P_1 (remaining 9). P_2 runs 1–2 and finishes.
- t = 2: P_3 arrives (priority 4, lower than P_1's 3). P_1 resumes.
- t = 3: P_4 arrives (priority 2, higher than P_1's 3). P_4 preempts P_1 (remaining 8). P_4 runs 3–4 and finishes.
- t = 4: ready queue: P_1 (priority 3, remaining 8) and P_3 (priority 4). P_1 has the higher priority: it runs 4–12 and finishes.

- $t = 12$: P_3 runs 12–14 and finishes.



Gantt chart for preemptive priority scheduling.

Step 2 — Criteria table.

Process	AT	BT	CT	TAT = CT − AT
P_1	0	10	12	12
P_2	1	1	2	1
P_3	2	2	14	12
P_4	3	1	4	1

$$\text{Average TAT} = \frac{12 + 1 + 12 + 1}{4} = \frac{26}{4} = 6.5 \text{ ms}$$

Conclusion. Urgent tasks (P_2, P_4) are served almost immediately — exactly what a hospital system needs. But P_3, with the lowest priority, waits 10 ms before starting (it arrives at $t = 2$ and first runs at $t = 12$): without **ageing**, such a process could starve indefinitely if high-priority tasks keep arriving.

7.6 Method 6 — Synchronising Processes with Semaphores

METHOD **Solving a Synchronisation Problem with Semaphores**

1. Identify the critical sections: the code regions where shared resources (variables, files, printer) are accessed and which must not be executed by two processes at once.
2. Choose the semaphores: a **binary semaphore** (or mutex) initialised to 1 for mutual exclusion; a **counting semaphore** initialised to n for a resource with n identical instances (e.g. 3 printers).
3. Protect each critical section: `wait(S)` (or `P(S)`) before entering, `signal(S)` (or `V(S)`) after leaving.
4. Recall the semantics: `wait(S)` decrements S; if $S < 0$ the process blocks. `signal(S)` increments S; if processes are blocked, one is woken up.
5. Check the invariants: the semaphore's initial value must equal the number of pro-

cesses allowed simultaneously in the critical section; every `wait` must be matched by a `signal` on every execution path.

6. For producer–consumer problems, use two counting semaphores (`empty` = N, `full` = 0) plus a mutex for the buffer itself.

WORKED EXAMPLE 13 **Mutual Exclusion in a School Fees System**

Two clerks of a college in Bamenda simultaneously update the same shared variable `balance` (school fees account), each executing: read `balance`, add the payment, write `balance`.
1) Explain what can go wrong without synchronisation. 2) Write pseudocode using a semaphore that guarantees a correct update. 3) Trace the semaphore values if both processes try to enter at the same time.

SOLUTION

1) The race condition. Suppose `balance` = 50 000 FCFA, clerk A adds 10 000 and clerk B adds 25 000. Without protection the operations may interleave:

- A reads 50000; B reads 50000;
- A computes 50000 + 10000 = 60000 and writes 60000;
- B computes 50000 + 25000 = 75000 and writes 75000.

Final value: 75 000 instead of 85 000 — A's update is lost. This is a **race condition**: the result depends on the unpredictable order of execution. The three instructions form a **critical section**.
2) Solution with a binary semaphore. Declare `mutex`, a binary semaphore initialised to 1. Each clerk executes:

```
wait(mutex);

read balance;

balance := balance + payment;

write balance;

signal(mutex);
```

The `wait`/`signal` pair guarantees **mutual exclusion**: at most one process is inside the critical section at any time.
3) Trace of the semaphore. Recall: `wait` decrements and blocks the caller if the value becomes negative; `signal` increments and wakes a blocked process if any.

Step	Action	`mutex`	Effect
1	Initial state	1	critical section free
2	A executes `wait(mutex)`	0	A enters the section
3	B executes `wait(mutex)`	-1	B blocks
4	A updates and executes `signal`	0	B is woken up, enters
5	B executes `signal(mutex)`	1	section free again

Now the updates happen one after the other: A brings the balance to 60 000, then B reads 60 000 and writes 85 000. The final value is correct.
Remark. Forgetting the `signal` (for example on an error path) would leave `mutex` at 0 forever: every other process would block indefinitely — a classic programming error that leads to deadlock.

7.7 Method 7 — Detecting and Handling Deadlock

METHOD **Analysing a Deadlock Situation**

1. Recall the four necessary conditions (Coffman): **mutual exclusion**, **hold and wait**, **no preemption**, **circular wait**. A deadlock requires all four.
2. Model the situation with a **resource allocation graph** (RAG): circles for processes, rectangles for resources; an arrow resource → process means "allocated to", an arrow process → resource means "requests".
3. Detect: with one instance per resource, a deadlock exists if and only if the graph contains a cycle.
4. Prevent: design the system so that at least one condition can never hold (e.g. impose a global ordering of resources to break circular wait).
5. Avoid: before granting a request, check with the **Banker's algorithm** that the resulting state is safe (an order exists in which all processes can finish).
6. Recover: if deadlock occurs, kill a victim process or preempt its resources, then roll back.

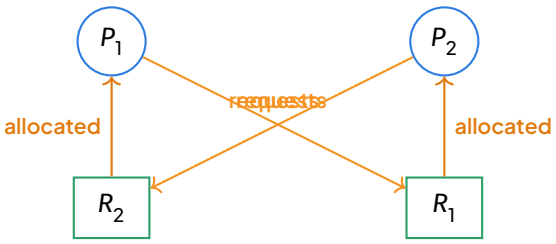
WORKED EXAMPLE 14 **Deadlock Detection with a Resource Graph**

A system has two processes P_1, P_2 and two single-instance resources R_1 (printer) and R_2 (scanner). The current state is: R_1 is allocated to P_1; R_2 is allocated to P_2; P_1 requests R_2; P_2

requests R_1.
1) Draw the resource allocation graph. 2) Is the system deadlocked? Justify using the four conditions. 3) Propose one prevention strategy and one recovery action.

SOLUTION

1) Resource allocation graph.



Resource allocation graph showing a cycle.

2) Deadlock analysis. The graph contains the cycle

$$P_1 \to R_1 \to P_2 \to R_2 \to P_1.$$

Since each resource has a single instance, a cycle is a sufficient condition for deadlock. Let us verify the four Coffman conditions:

- Mutual exclusion: a printer or scanner can serve only one process at a time. ✓
- Hold and wait: P_1 holds R_1 while waiting for R_2; P_2 holds R_2 while waiting for R_1. ✓
- No preemption: the OS cannot forcefully take the printer from P_1. ✓
- Circular wait: the cycle above shows each process waits for a resource held by the other. ✓

All four conditions hold: the system is deadlocked. Neither process can ever proceed.
3) Handling strategies.

- Prevention (break circular wait): impose a global order on resources, e.g. every process must request the printer (R_1) before the scanner (R_2). Then P_2 could not hold R_2 while requesting R_1, and the cycle becomes impossible.
- Recovery: choose a victim, e.g. abort P_2 (or preempt R_2 from it). P_1 then obtains R_2, finishes and releases R_1; P_2 is restarted later. The victim is usually chosen to minimise cost (lowest priority, least work done).

7.8 Method 8 — Integration: Comparing Algorithms and Choosing One

METHOD **Choosing and Justifying a Scheduling Policy (Integration)**

1. Compute the average waiting and turnaround times for each candidate algorithm on the given data — never compare algorithms from intuition alone.
2. Match the algorithm to the context: interactive systems need good response time (Round Robin); batch systems need throughput (SJF); urgent tasks need priorities; real-time systems need deadlines guarantees.
3. Discuss side effects: starvation (SJF, priority), convoy effect (FCFS), context-switch overhead (small quantum in RR).
4. Conclude in writing: state the chosen algorithm, the numerical evidence, and one limitation — this three-part argument is what GCE examiners reward.

WORKED EXAMPLE 15 **Integration Activity — The GCE Board Results Server**

The results-processing server of an examination board receives three batch jobs at the same instant $t = 0$:

Job	Burst time (s)
J_1	24
J_2	3
J_3	3

1) Compute the average waiting time under FCFS (order J_1, J_2, J_3) and under SJF. 2) The board also runs an interactive terminal service on the same machine. Which algorithm would you recommend for the combined workload, and why? 3) State one risk of your recommendation and how to limit it.

SOLUTION

1) Numerical comparison. All jobs arrive at $t = 0$, so WT of the first job is 0 and each following job waits for the sum of the preceding bursts.
FCFS, order J_1, J_2, J_3:

- J_1: runs 0–24, WT = 0;
- J_2: runs 24–27, WT = 24;
- J_3: runs 27–30, WT = 27.

$$\text{Average WT (FCFS)} = \frac{0 + 24 + 27}{3} = \frac{51}{3} = 17\ \text{s}.$$

SJF: shortest first: J_2 and J_3 (3 s each, tie broken by arrival order), then J_1.

- J_2: runs 0–3, WT = 0;

- J_3: runs 3–6, WT = 3;
- J_1: runs 6–30, WT = 6.

$$\text{Average WT (SJF)} = \frac{0 + 3 + 6}{3} = \frac{9}{3} = 3\ \text{s}.$$

SJF reduces the average waiting time from 17 s to 3 s — a dramatic improvement, caused by eliminating the **convoy effect** (short jobs stuck behind the 24 s job).

2) Recommendation. For a combined workload (batch jobs plus interactive terminals), I recommend Round Robin (or a multilevel feedback queue). Justification:

- Interactive users need a response time of a fraction of a second; under SJF or FCFS, a terminal user arriving behind the 24 s job would wait up to 24 s before the first response — unacceptable.
- Round Robin gives every process the CPU within at most $(n-1) \times q$ time units, so terminals stay responsive while batch jobs progress steadily.
- Numerically, SJF is optimal for average waiting time, but it optimises the wrong criterion for interactive work.

3) Risk and mitigation. Round Robin's risk is context-switch overhead: if the quantum is too small (say $q = 1$ ms for jobs measured in seconds), the CPU spends a significant fraction of its time switching instead of computing. The remedy is to choose a quantum large compared with the switch cost but small compared with typical interactive bursts (commonly 10–100 ms), or to use a multilevel feedback queue that gives short quanta to interactive processes and long ones to batch jobs.

Final argument (as expected at the GCE). SJF is numerically the best for pure batch (3 s vs 17 s average wait), but the presence of interactive users makes response time the dominant criterion; therefore Round Robin with a well-chosen quantum is the appropriate policy, its overhead being controlled by the choice of q.

8 EXERCISES

8.1 I check my knowledge

EXERCISE 10 **Multiple choice questions**

For each question, choose the single correct answer.

1. A **process** is:
   (a) a program stored on the hard disk;
   (b) a program in execution, with its own state;
   (c) a hardware component of the CPU;
   (d) a file containing source code.
2. The information about a process (state, program counter, registers, open files) is stored by the operating system in:
   (a) the cache memory;
   (b) the **Process Control Block** (PCB);
   (c) the page table only;
   (d) the boot sector.
3. Which scheduling algorithm can cause **starvation** of long processes?
   (a) FCFS;
   (b) Round Robin;
   (c) Shortest Job First (SJF);
   (d) none of them.
4. A deadlock can occur only if the following number of Coffman conditions hold simultaneously:
   (a) one;
   (b) two;
   (c) three;
   (d) four.

EXERCISE 11 **True or false — justify**

Say whether each statement is true or false, and justify your answer in one or two sentences.

1. In a non-preemptive scheduler, a running process can be forced to leave the CPU before it terminates or blocks.
2. A context switch consumes CPU time during which no useful user work is done.
3. A binary semaphore initialised to 1 can be used to guarantee mutual exclusion on a critical section.
4. If a Resource Allocation Graph contains a cycle, then a deadlock necessarily exists, whatever the number of instances of each resource.

EXERCISE 12 **Definitions**

Give a precise definition of each of the following terms, in your own words:

1. **turnaround time** of a process;
2. **waiting time** of a process;
3. **response time** of a process;
4. **race condition**;
5. **deadlock**.

EXERCISE 13 **Direct calculation**

Three processes arrive at time 0 in the order P_1, P_2, P_3, with burst times 8, 4 and 2 ms respectively.

1. Compute the waiting time and the turnaround time of each process under FCFS scheduling.
2. Compute the average turnaround time.
3. Repeat the two previous questions assuming the burst times are executed in the order P_3, P_2, P_1. What do you observe?

8.2 I practise

EXERCISE 14 FCFS and SJF Gantt charts

Four processes are submitted to a single-CPU system:

Process	Arrival time (ms)	Burst time (ms)
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

1. Draw the Gantt chart for FCFS scheduling and compute the average waiting time.
2. Draw the Gantt chart for non-preemptive SJF scheduling and compute the average waiting time.
3. Which algorithm performs better on this example? Justify.

EXERCISE 15 Round Robin

Five processes arrive at time 0 in the order $P_1, \dots, P_5$ with burst times 10, 3, 6, 8 and 5 ms. The system uses Round Robin with a quantum $q = 4$ ms.

1. Draw the Gantt chart.
2. Compute the response time of each process.
3. Compute the average turnaround time.
4. Without redoing the whole computation, discuss the effect of halving the quantum to $q = 2$ ms on the response time and on the overhead due to context switches.

EXERCISE 16 The lost increment

Two processes *A* and *B* share a variable `count` initialised to 0. Each executes, once, the high-level instruction `count := count + 1`, which the machine carries out in three steps: `LOAD` (read `count` into a register), `ADD 1`, `STORE` (write the register back).

1. Exhibit an interleaving of the six machine instructions for which the final value of `count` is 1 instead of 2.
2. Name the phenomenon observed.
3. Correct the situation using a binary semaphore `mutex`: give its initial value and the pseudocode of each process.

EXERCISE 17 **Producer–consumer with a bounded buffer**

A producer process and a consumer process share a circular buffer of 5 slots. The producer must never write into a full buffer and the consumer must never read from an empty buffer.

1. Give the semaphores needed and their initial values.
2. Write the complete pseudocode of the producer and of the consumer using the operations `wait` and `signal`.
3. Explain why the order of the two `wait` operations in each process is critical: what happens if the mutual-exclusion semaphore is taken before the counting semaphore?

8.3 I prepare for the GCE

EXERCISE 18 **Scheduling comparison — GCE style**

A computer centre in Yaoundé runs a batch system with one CPU. Four jobs are submitted as follows:

Job	Arrival time (ms)	Burst time (ms)
J_1	0	8
J_2	1	4
J_3	2	9
J_4	3	5

1. Define the terms waiting time and turnaround time of a process. (2 marks)
2. Draw the Gantt chart and compute the average turnaround time and the average waiting time for each of the following algorithms:
   (a) FCFS; (4 marks)
   (b) non-preemptive SJF; (4 marks)
   (c) preemptive SJF (Shortest Remaining Time First). (5 marks)
3. State which algorithm gives the best average waiting time on this example, and justify why this algorithm is optimal with respect to average waiting time. (2 marks)
4. Give one reason why this optimal algorithm is difficult to use in practice. (1 mark)
5. The centre decides to switch to Round Robin with quantum $q = 3$ ms. Draw the new Gantt chart and compute the response time of each job. (4 marks)

EXERCISE 19 **Banker's algorithm and Resource Allocation Graph — GCE style**

A system has three resource types R_1, R_2, R_3 and four processes. The current state is:

	Allocation			Max		
Process	R_1	R_2	R_3	R_1	R_2	R_3
P_1	0	1	0	7	5	3
P_2	2	0	0	3	2	2
P_3	3	0	2	9	0	2
P_4	2	1	1	2	2	2

The vector of available resources is Available = (3, 3, 2).

1. Compute the Need matrix. (2 marks)
2. Apply the Banker's safety algorithm to determine whether the system is in a safe state. If it is, exhibit a safe sequence; if it is not, justify carefully. (6 marks)
3. P_2 now requests one additional instance of R_1 and one of R_2. Using the Banker's algorithm, decide whether this request can be granted immediately. (4 marks)
4. State the four Coffman conditions necessary for a deadlock. (4 marks)
5. In another system, P_1 holds one instance of R_1 and requests R_2; P_2 holds one instance of R_2 and requests R_3; P_3 holds one instance of R_3 and requests R_1. Each resource type has exactly one instance. Draw the Resource Allocation Graph, state whether a deadlock exists, and justify your answer using the Coffman conditions. (4 marks)

EXERCISE 20 **Structured essay — processes, synchronisation and scheduling trade-offs**

1. Define the following terms: process, Process Control Block, context switch. (3 marks)
2. Explain why a context switch is considered pure overhead, and state two consequences of this overhead on the design of a scheduling policy. (3 marks)
3. A bank in Douala equips its branches with interactive customer terminals connected to a central server. Discuss the trade-offs between preemptive and non-preemptive scheduling for such a system. Your discussion should address: responsiveness to interactive users, overhead, starvation, and predictability for long batch jobs such as

end-of-day account processing. Conclude by recommending a scheduling discipline and justifying your choice. (8 marks)

4. The server runs two cooperating processes that both update a shared counter recording the number of transactions of the day. Explain, with an example of instruction interleaving, how the final value of the counter can be wrong, and show how a binary semaphore solves the problem. (6 marks)

9 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. Answer (b). A process is a program in execution, together with its current state (program counter, registers, stack, open files). A program on disk (a) is a passive entity; it becomes a process only when loaded and executed.
2. Answer (b). All the information the operating system needs about a process is grouped in the **Process Control Block** (PCB), one per process. The PCB is what makes a context switch possible: the state of the suspended process is saved in its PCB and restored later.
3. Answer (c). Under **SJF**, a long job can be postponed indefinitely if short jobs keep arriving: this is **starvation**. FCFS serves jobs in arrival order and Round Robin gives every process a quantum, so neither can starve a process permanently.
4. Answer (d). A deadlock requires the four Coffman conditions to hold simultaneously: mutual exclusion, hold and wait, no preemption, and circular wait. If any single one is broken, deadlock is impossible.

SOLUTION **Exercise 2 — True or false**

1. False. By definition, a non-preemptive scheduler lets the running process keep the CPU until it terminates or blocks voluntarily (e.g. for an I/O request). Forcing it out before that is precisely what preemption means.
2. True. During a **context switch** the CPU saves the PCB of the old process and loads the PCB of the new one; no user instruction is executed. It is pure overhead, which is why the quantum in Round Robin must not be too small.
3. True. A binary semaphore `mutex` initialised to 1 guarantees mutual exclusion: the first process that executes `wait(mutex)` enters its critical section, and any other process executing `wait(mutex)` is blocked until `signal(mutex)` is called.
4. False. A cycle in a Resource Allocation Graph is a necessary condition for deadlock only when each resource type has a single instance. If resources have several instances, a cycle may exist without deadlock, because a process in the cycle may be completed using an instance held by a process outside the cycle.

SOLUTION **Exercise 3 — Definitions**

1. Turnaround time: the total time elapsed between the submission (arrival) of a process and its completion: $T_{\text{turn}} = t_{\text{completion}} - t_{\text{arrival}}$. It includes waiting, CPU execution and I/O.
2. Waiting time: the total time a process spends in the ready queue, i.e. waiting for the CPU: $T_{\text{wait}} = T_{\text{turn}} - T_{\text{burst}}$ (for a process that does no I/O).
3. Response time: the time between the arrival of a process and the first moment it receives the CPU: $T_{\text{resp}} = t_{\text{first run}} - t_{\text{arrival}}$. It measures the reactivity perceived by an interactive user.
4. Race condition: a situation in which several processes access shared data concurrently and the final result depends on the exact order (interleaving) of their machine instructions, so that some executions give an incorrect result.
5. Deadlock: a situation in which a set of processes are all blocked, each one waiting for a resource held by another process of the set, so that none of them can ever proceed without external intervention.

SOLUTION **Exercise 4 — Direct calculation**

(a) FCFS in the order P_1, P_2, P_3. All processes arrive at time 0, so waiting time of a process is the sum of the burst times of the processes executed before it.

Process	Burst	Waiting time	Turnaround time
P_1	8	0	0 + 8 = 8
P_2	4	8	8 + 4 = 12
P_3	2	8 + 4 = 12	12 + 2 = 14

(b) Average turnaround time:

$$\bar{T}_{\text{turn}} = \frac{8 + 12 + 14}{3} = \frac{34}{3} \approx 11.33\ \text{ms}.$$

(c) Order P_3, P_2, P_1:

Process	Burst	Waiting time	Turnaround time
P_3	2	0	2
P_2	4	2	6
P_1	8	2 + 4 = 6	14

$$\bar{T}_{\text{turn}} = \frac{2 + 6 + 14}{3} = \frac{22}{3} \approx 7.33\ \text{ms}.$$

Observation: the average turnaround time drops from 11.33 ms to 7.33 ms simply by running the shortest jobs first. This is exactly the idea behind the **SJF** algorithm: serving short jobs early reduces the time that many processes spend waiting behind a long job.

SOLUTION **Exercise 5 — FCFS and SJF Gantt charts**

(a) FCFS. Processes are served in arrival order P_1, P_2, P_3, P_4:

P_1	P_2	P_3	P_4
0 – 7	7 – 11	11 – 12	12 – 16

Waiting times: P_1 : 0; P_2 : 7 - 2 = 5; P_3 : 11 - 4 = 7; P_4 : 12 - 5 = 7.

$$\bar{T}_{\text{wait}} = \frac{0 + 5 + 7 + 7}{4} = \frac{19}{4} = 4.75 \text{ ms}.$$

(b) Non-preemptive SJF. At $t = 0$ only P_1 has arrived, so it runs first (non-preemptive: it cannot be interrupted). At $t = 7$, the ready processes are P_2 (burst 4), P_3 (burst 1), P_4 (burst 4): the shortest is P_3, then P_2 and P_4 (tie broken by arrival order).

P_1	P_3	P_2	P_4
0 – 7	7 – 8	8 – 12	12 – 16

Waiting times: P_1 : 0; P_3 : 7 - 4 = 3; P_2 : 8 - 2 = 6; P_4 : 12 - 5 = 7.

$$\bar{T}_{\text{wait}} = \frac{0 + 3 + 6 + 7}{4} = \frac{16}{4} = 4 \text{ ms}.$$

(c) SJF performs better here (4 ms against 4.75 ms average waiting time). This illustrates the general result: SJF gives the minimum average waiting time for a given set of jobs, because every job that runs early adds its burst to the waiting time of fewer processes.

SOLUTION **Exercise 6 — Round Robin**

(a) Gantt chart (quantum $q = 4$ ms; a process that finishes its quantum with remaining burst goes to the back of the queue):

P_1	P_2	P_3	P_4	P_5	P_1	P_3	P_4	P_1	P_5
0–4	4–7	7–11	11–15	15–19	19–23	23–25	25–29	29–31	31–32

(Remaining bursts: after the first round P_1 : 6, P_2 : 0, P_3 : 2, P_4 : 4, P_5 : 1; then P_1 uses 4 more, P_3 finishes with 2, P_4 finishes with 4, P_1 finishes with 2, P_5 finishes with 1.) (b) Response times (time of first allocation of the CPU, since all arrive at 0):

Process	P_1	P_2	P_3	P_4	P_5
Response time (ms)	0	4	7	11	15

(c) Turnaround times (completion times, since arrival is at 0): P_1 : 31, P_2 : 7, P_3 : 25, P_4 : 29, P_5 : 32.

$$\bar{T}_{\text{turn}} = \frac{31 + 7 + 25 + 29 + 32}{5} = \frac{124}{5} = 24.8 \text{ ms}.$$

(d) Effect of $q = 2$ ms. Each process is reached by the scheduler sooner after its arrival, so the response time decreases: the system feels more reactive. However, the number of quanta (and therefore of **context switches**) roughly doubles; since each context switch is pure overhead, a larger fraction of CPU time is wasted, which increases the total completion time of the batch. The quantum must therefore be chosen as a compromise: large enough to keep overhead small, small enough to keep the system responsive.

SOLUTION **Exercise 7 — The lost increment**

(a) Let R_A and R_B be the registers of A and B. Consider the interleaving:

1. A: `LOAD` $R_A \leftarrow$ `count` ($R_A = 0$)
2. B: `LOAD` $R_B \leftarrow$ `count` ($R_B = 0$)
3. A: `ADD 1` ($R_A = 1$)
4. A: `STORE count` $\leftarrow R_A$ (`count` = 1)
5. B: `ADD 1` ($R_B = 1$)
6. B: `STORE count` $\leftarrow R_B$ (`count` = 1)

Both processes read the value 0 before either writes back, so the final value is 1 instead of 2: one increment is lost. (b) This is a **race condition** (more precisely a lost update): the result depends on the interleaving of the instructions of the two processes. (c) Use a binary semaphore `mutex` initialised to 1; the update becomes a critical section:

- `Process A (identically B):`
- `wait(mutex);`
- `count := count + 1;`
- `signal(mutex);`

Now whichever process executes `wait(mutex)` first completes all three machine steps before the other can read `count`, so the final value is always 2.

SOLUTION **Exercise 8 — Producer–consumer with a bounded buffer**

(a) Semaphores:

- `empty` = 5: counts the free slots (initially the whole buffer is empty);
- `full` = 0: counts the occupied slots;
- `mutex` = 1: binary semaphore protecting the buffer itself (mutual exclusion).

(b) Pseudocode:

```
Producer: loop forever
    produce an item;
    wait(empty);        % wait for a free slot
    wait(mutex);        % enter critical section
    insert item into buffer;
    signal(mutex);
    signal(full);        % one more occupied slot
Consumer: loop forever
    wait(full);         % wait for an occupied slot
    wait(mutex);
    remove item from buffer;
    signal(mutex);
    signal(empty);       % one more free slot
    consume the item;
```

(c) Why the order matters. Suppose the producer took `mutex` before `empty`. If the buffer is full, the producer blocks on `wait(empty)` while still holding `mutex`. The consumer, which is the only process able to free a slot, then blocks on `wait(mutex)`. Each process waits for the other forever: a **deadlock**. The rule is therefore: always wait on the resource-counting (synchronisation) semaphore first, and on the mutual-exclusion semaphore last.

SOLUTION **Exercise 9 — Scheduling comparison (GCE style)**

(a) Definitions (2 marks — 1 per definition). Waiting time: total time a process spends in the ready queue waiting for the CPU. Turnaround time: total time from arrival to completion, $T_{\text{turn}} = t_{\text{completion}} - t_{\text{arrival}}$.

(b) (i) FCFS (4 marks). Order J_1, J_2, J_3, J_4:

J_1	J_2	J_3	J_4
0–8	8–12	12–21	21–26

Turnaround: J_1 : 8, J_2 : 12 - 1 = 11, J_3 : 21 - 2 = 19, J_4 : 26 - 3 = 23; average = 61/4 = 15.25 ms. Waiting: 0, 7, 10, 18; average = 35/4 = 8.75 ms.
(ii) Non-preemptive SJF (4 marks). At t = 0 only J_1 is present, so it runs to completion. At t = 8: J_2 (4), J_4 (5), J_3 (9) are ready, in that SJF order.

J_1	J_2	J_4	J_3
0–8	8–12	12–17	17–26

Turnaround: J_1 : 8, J_2 : 11, J_4 : 14, J_3 : 24; average = 57/4 = 14.25 ms. Waiting: 0, 7, 9, 15; average = 31/4 = 7.75 ms.
(iii) Preemptive SJF / SRTF (5 marks). At t = 1, J_2 (burst 4) arrives while J_1 has 7 ms remaining: J_2 preempts. At t = 2 and t = 3, J_3 (9) and J_4 (5) arrive but J_2's remaining time (3, then 2) stays the smallest. At t = 5, J_2 finishes; the remaining times are J_4 : 5, J_1 : 7, J_3 : 9, so J_4 runs to completion, then J_1, then J_3.

J_1	J_2	J_4	J_1	J_3
0–1	1–5	5–10	10–17	17–26

Turnaround: J_1 : 17, J_2 : 5 - 1 = 4, J_3 : 26 - 2 = 24, J_4 : 10 - 3 = 7; average = 52/4 = 13 ms. Waiting: J_1 : 17 - 8 = 9, J_2 : 0, J_3 : 24 - 9 = 15, J_4 : 7 - 5 = 2; average = 26/4 = 6.5 ms.
(c) (2 marks). SRTF gives the best average waiting time (6.5 ms). Justification: SJF/SRTF is optimal with respect to average waiting time because always running the job with the smallest (remaining) burst minimises the sum of the waiting times — any exchange that places a longer job before a shorter one increases the total.
(d) (1 mark). In practice the CPU burst of a job is not known in advance (it can only be estimated); moreover long jobs may suffer **starvation** if short jobs keep arriving.
(e) Round Robin, q = 3 (4 marks). Ready queue discipline: newly arrived jobs join the queue, then the preempted job is requeued behind them.

J_1	J_2	J_3	J_4	J_1	J_2	J_3	J_4	J_1
0–3	3–6	6–9	9–12	12–15	15–16	16–19	19–21	21–23

then J_3 runs 23–26 and finishes. (Remaining bursts: after first pass J_1 : 5, J_2 : 1, J_3 : 6, J_4 : 2.) Response times ($t_{\text{first run}} - t_{\text{arrival}}$): J_1 : 0 - 0 = 0 ms; J_2 : 3 - 1 = 2 ms; J_3 : 6 - 2 = 4 ms; J_4 : 9 - 3 = 6 ms.
Examiner's expectations: a correct Gantt chart with labelled time boundaries (essential for the method marks), waiting/turnaround computed from the chart, correct averages with units, and for (c) the keyword optimal linked to shortest-job-first reasoning.

SOLUTION **Exercise 10 — Banker's algorithm and RAG (GCE style)**

(a) Need matrix (2 marks). Need = Max - Allocation:

Process	R_1	R_2	R_3
P_1	7	4	3
P_2	1	2	2
P_3	6	0	0
P_4	0	1	1

(b) Safety algorithm (6 marks). Work = (3, 3, 2).

1. P_2: Need $(1, 2, 2) \le (3, 3, 2)$ ✓ Work ← $(3, 3, 2) + (2, 0, 0) = (5, 3, 2)$.
2. P_4: Need $(0, 1, 1) \le (5, 3, 2)$ ✓ Work ← $(5, 3, 2) + (2, 1, 1) = (7, 4, 3)$.
3. P_1: Need $(7, 4, 3) \le (7, 4, 3)$ ✓ Work ← $(7, 4, 3) + (0, 1, 0) = (7, 5, 3)$.
4. P_3: Need $(6, 0, 0) \le (7, 5, 3)$ ✓ Work ← $(7, 5, 3) + (3, 0, 2) = (10, 5, 5)$.

All processes can finish: the system is in a **safe state**, with safe sequence $\langle P_2, P_4, P_1, P_3 \rangle$ (other safe sequences exist, e.g. $\langle P_4, P_2, P_1, P_3 \rangle$).

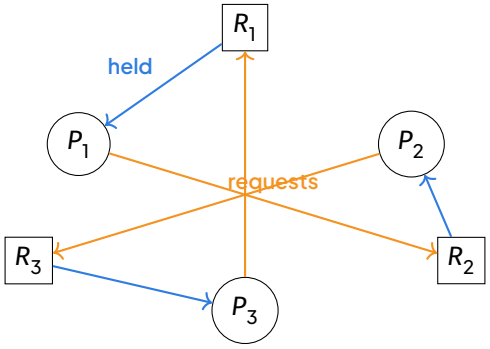
(c) Request (1, 1, 0) by P_2 (4 marks).

1. Request $(1, 1, 0) \le \text{Need}_{P_2} = (1, 2, 2)$ ✓ (the request is legal).
2. Request $(1, 1, 0) \le$ Available $= (3, 3, 2)$ ✓ (resources exist).
3. Pretend to grant: Available = (2, 2, 2); $\text{Alloc}_{P_2} = (3, 1, 0)$; $\text{Need}_{P_2} = (0, 1, 2)$.
4. Re-run safety: P_2: $(0, 1, 2) \le (2, 2, 2)$ ✓, Work = $(5, 3, 2)$; P_4: $(0, 1, 1) \le (5, 3, 2)$ ✓, Work = $(7, 4, 3)$; P_1: $(7, 4, 3) \le (7, 4, 3)$ ✓, Work = $(7, 5, 3)$; P_3: $(6, 0, 0) \le (7, 5, 3)$ ✓, Work = $(10, 5, 5)$.

The resulting state is still safe (sequence $\langle P_2, P_4, P_1, P_3 \rangle$), so the request can be granted immediately.

(d) Coffman conditions (4 marks — 1 each). (i) Mutual exclusion: at least one resource is held in a non-sharable mode; (ii) Hold and wait: a process holds resources while requesting others; (iii) No preemption: resources cannot be forcibly taken, only released voluntarily; (iv) Circular wait: a cycle of processes each waiting for a resource held by the next.

(e) Resource Allocation Graph (4 marks).



Resource Allocation Graph: assignment edges $R_i \rightarrow P_i$ (blue) and request edges $P_i \rightarrow R_j$ (orange).

The graph contains the cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$. Since each resource type has exactly one instance, a cycle here is a sufficient condition: a deadlock exists. Justification via Coffman: each resource is non-sharable (mutual exclusion); each process holds one resource while requesting another (hold and wait); resources cannot be preempted (no preemption); and the cycle above is a circular wait. All four conditions hold simultaneously, so none of P_1, P_2, P_3 can ever proceed.
Examiner's expectations: the Need matrix computed as Max - Allocation (not the reverse); the safety algorithm shown step by step with the evolving Work vector; for (c), the three checks (request ≤ Need, request ≤ Available, safety of the hypothetical state) — simply saying "resources are available" earns no credit; for (e), correct orientation of the two edge types.

SOLUTION **Exercise 11 — Structured essay**

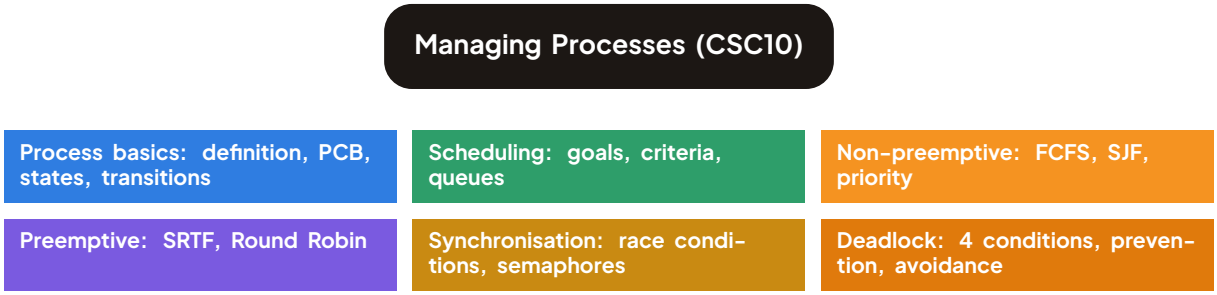
(a) Definitions (3 marks — 1 each). A process is a program in execution, together with its current state (program counter, registers, stack, data, open files). The Process Control Block (PCB) is the data structure, maintained by the operating system, that stores all the information about one process: its identifier, state, program counter, register values, memory and open-file information. A context switch is the operation by which the CPU passes from one process to another: the state of the outgoing process is saved in its PCB and the state of the incoming process is restored from its own PCB.
(b) Context switch as overhead (3 marks). During a context switch the CPU executes only operating-system code (saving and restoring registers, updating queues); no user program progresses: it is pure overhead (1 mark). Consequences on scheduling design (any two, 1 mark each): the Round Robin quantum must not be too small, otherwise the proportion of time wasted in switches becomes unacceptable; a scheduler should avoid unnecessary preemptions, so non-preemptive or weakly preemptive policies may be preferred for batch work; the number of processes competing for the CPU (degree of multi-

programming) must be controlled.
(c) Preemptive vs non-preemptive scheduling for the bank (8 marks). Indicative mark scheme: 2 marks per correctly discussed criterion.

- Responsiveness. Interactive tellers and customers at terminals need answers within a second or so. A non-preemptive scheduler would let a long job (e.g. printing statements) monopolise the CPU and freeze all terminals; a preemptive scheduler (Round Robin or priority with preemption) interrupts long jobs and gives each interactive session frequent slices of CPU, guaranteeing a short **response time**.
- Overhead. Preemption costs a context switch each time; with a small quantum the server spends a noticeable fraction of its power switching instead of computing. Non-preemptive scheduling has almost no such overhead.
- Starvation. Under preemptive priority or SJF-like policies, long batch jobs may be perpetually postponed by arriving interactive requests (starvation); a remedy is aging (gradually raising the priority of waiting processes). FCFS non-preemptive scheduling has no starvation but terrible response times (the convoy effect).
- Predictability. End-of-day account processing must finish before the next morning; non-preemptive execution of these batch jobs (e.g. at night) gives predictable completion times, whereas preemption makes their duration depend on the interactive load.

Conclusion (justified recommendation): a preemptive discipline such as Round Robin with a moderate quantum (or multilevel queues: interactive jobs in a high-priority RR queue, batch jobs in a lower FCFS queue scheduled mainly at night) is recommended, because for a banking terminal system the perceived responsiveness is the critical criterion, and the modest context-switch overhead is an acceptable price; aging protects the batch jobs from starvation.
(d) Shared counter and semaphore (6 marks). The update `counter := counter + 1` compiles to `LOAD`, `ADD 1`, `STORE`. With `counter` = 100, consider the interleaving: process *A* executes `LOAD` (reads 100); the scheduler preempts *A*; process *B* executes `LOAD` (reads 100), `ADD 1`, `STORE` (writes 101); *A* resumes with `ADD 1` (its register becomes 101) and `STORE` (writes 101). Two transactions were recorded but the counter increased only by 1: a **race condition** / lost update (3 marks: 1 for the three machine steps, 2 for a correct interleaving with explanation). Solution: a binary semaphore `mutex` initialised to 1; each process executes `wait(mutex)` before the update and `signal(mutex)` after it (2 marks). The critical section is then executed atomically: the second process cannot read the counter until the first has written it back, so the final value is always correct (1 mark).

10 SUMMARY SHEET



Mind map of the chapter: from the life of a process to deadlock handling.

FICHE BILAN **L'essentiel à connaître pour l'examen**

1. Key definitions

- A **process** is a program in execution, with its own code, data, stack and context. A program is passive; a process is active.
- The **Process Control Block** (PCB) is the data structure the operating system keeps for each process: process ID, state, program counter, register values, memory information, open files, scheduling data.
- The main **process states** are: New, Ready, Running, Blocked (Waiting), Terminated. Transitions: admit (New → Ready), dispatch (Ready → Running), interrupt/time-out (Running → Ready), I/O request (Running → Blocked), I/O completion (Blocked → Ready), exit (Running → Terminated).
- A **context switch** saves the state of the current process in its PCB and loads the state of the next one; it is pure overhead.
- **CPU scheduling** decides which ready process gets the processor. **Non-preemptive**: a running process keeps the CPU until it finishes or blocks. **Preemptive**: the OS may take the CPU back (interrupt, higher priority, end of time quantum).
- A **race condition** occurs when several processes access shared data concurrently and the result depends on the order of execution. A **critical section** is the code region accessing the shared resource; it must run in **mutual exclusion**.
- A **semaphore** is an integer variable accessed only through the atomic operations `wait(P)` and `signal(V)`. A binary semaphore (mutex) values 0 or 1.
- A **deadlock** is a situation where a set of processes each wait for a resource held by another member of the set, so none can ever proceed.

2. Formulas and results to know

- Turnaround time: $T_{\text{turn}} = T_{\text{completion}} - T_{\text{arrival}}$.
- Waiting time: $T_{\text{wait}} = T_{\text{turn}} - T_{\text{burst}}$.
- Response time: $T_{\text{resp}} = T_{\text{first run}} - T_{\text{arrival}}$.
- CPU utilisation $= \dfrac{\text{busy time}}{\text{total time}} \times 100\,\%$; throughput $= \dfrac{\text{number of processes completed}}{\text{time}}$.
- FCFS: average waiting time depends heavily on arrival order (convoy effect). SJF/SRTF give the minimum average waiting time for a given set of bursts (no proof required at GCE, but you must be able to exploit it).
- Round Robin with quantum q: a process with burst B needs $\lceil B/q \rceil$ slices; small $q \Rightarrow$ good response but many context switches; large $q \Rightarrow$ behaves like FCFS.
- Coffman's four **necessary conditions** for deadlock: mutual exclusion, hold and wait, no preemption, circular wait. All four must hold simultaneously.

3. Methods to master

1. Gantt chart method: list processes with arrival and burst times; draw the CPU timeline; read off completion times; compute turnaround, waiting and response times; average over all processes.
2. Choosing the running process: FCFS → earliest arrival; SJF → shortest burst among arrived processes; SRTF → shortest remaining time, rechecked at every arrival; RR → circular ready queue, quantum q; priority → highest priority (state the convention: small number = high priority).
3. Protecting a critical section: `wait(mutex)` before entering, critical section, `signal(mutex)` after leaving. Producer–consumer: semaphores `empty` (initialised to N), `full` (initialised to 0), plus `mutex`.
4. Deadlock handling: prevention (break one Coffman condition, e.g. impose a global resource ordering), avoidance (Banker's algorithm: grant only if the resulting state is safe), detection and recovery (resource allocation graph, kill or roll back a victim).
5. Banker's safety check: a state is safe if there exists an order in which every process can obtain its remaining need (Need = Max − Allocation) and finish.

4. Common mistakes (examiner's warnings)

- Confusing waiting time and turnaround time: always subtract the burst for waiting time.
- In SRTF, forgetting to re-examine remaining times when a new process arrives.
- In Round Robin, forgetting that a process arriving during a quantum joins the back of the queue, and that a process finishing exactly at the quantum boundary leaves before the new arrival is enqueued (state your convention clearly).

- Believing SJF is practical: the burst time is unknown in advance; it is only estimated.
- Writing `wait`/`signal` in the wrong order, or placing them around the wrong code region: this destroys mutual exclusion or causes deadlock.
- Thinking one Coffman condition suffices: deadlock requires all four.

5. GCE tips

- Always draw the Gantt chart first: every mark for averages follows from it. Show the formula, the substitution, then the result.
- Read the question for the tie-breaking rule (usually FCFS among equals) and the priority convention.
- Keep one decimal place (or the requested precision) in averages, e.g. 6.5 ms, not 6.499 ...
- For synchronisation questions, name the shared resource, identify the critical section, then justify the semaphore initial values.
- For deadlock questions, quote the four Coffman conditions explicitly and say which one your proposed solution removes.
- Manage your time: a full scheduling table (FCFS, SJF, RR) is a classic 8–10 mark question; practise until a Gantt chart takes under five minutes.





Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+